

claude-windows / 02b9e9c9-3806-45ca-9b24-2ecc35a81efd

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\claude\projects\C--Users-avidu-Projects-muneem\02b9e9c9-3806-45ca-9b24-2ecc35a81efd\subagents\workflows\wf_27df53c8-8e0\agent-ae5557bfc6f68eb31.jsonl`  
- SHA-256: `2b2478a6385243885cafee97d86391341e9032b81926cc668bda89fdbcbb729a`  
- Source modified: `2026-06-10T16:20:27+00:00`  
- Imported at: `2026-07-05T16:48:21+00:00`  
- project: `wf_27df53c8-8e0`  
- session_id: `02b9e9c9-3806-45ca-9b24-2ecc35a81efd`
```

Transcript

1. user (2026-06-10T16:13:27.795Z)

You are auditing TEST QUALITY and CI for muneem (C:/Users/avidu/Projects/muneem, Python, src layout, pytest). The project claims: 80% coverage gate, mypy + ruff in CI, schema-lock + DB-invariant tripwires, license allowlist, Windows+Linux CI matrix, offline-only default suite (real statements are local-only), property fuzz on the verifier.

SENSITIVE-DATA RULES (mandatory, highest priority):

- testdata/aubank-statements/, testdata/axis-statements/, testdata/hdfc-statements/, testdata/snapshots/, and testdata/credentials/ in the muneem repo contain the user's REAL financial documents and a real OAuth client file. Do NOT open/read the PDFs. You MAY check file existence and git-tracking status (git ls-files, git log --all -- <path>).
- If you must inspect testdata/snapshots/*.json or testdata/credentials/*.json to assess sensitivity, read at most the JSON key structure and NEVER quote values (no account numbers, balances, person names, client secrets, transaction amounts) in your output.
- Never include sensitive values in your findings. Refer to them abstractly ("a real account number", "a client_secret value").
- Do not use WebSearch/WebFetch and do not send any repo content to any external service.

Do this:

1. Read .github/workflows/ci.yml, Makefile, pyproject.toml, .githubooks/* ? map what actually runs in CI vs locally vs not at all.
2. Read tests/ broadly (you do not need every line, but understand each file's purpose): conftest.py autouse fixtures (keyring isolation claim), test_db_invariants.py (schema lock), test_repo_hygiene.py, test_regression_synth.py + synthpdf.py (synthetic full-pipeline), test_verifier_properties.py, the local-only real-statement tests (tests/parsers/test_local*.py ? how are they skipped in CI?), test_promo_regression.py.
3. RUN the offline suite yourself and report the result: use 'C:/Users/avidu/Projects/muneem/.venv/Scripts/python.exe -m pytest' from C:/Users/avidu/Projects/muneem (check Makefile/pyproject for the right invocation and markers; if the venv lacks dev deps, report that instead of installing anything). Capture pass/fail counts and runtime. If tests fail, quote the failures (non-sensitive parts only).
4. Assess: what core behavior has NO test (key rotation failure paths? recovery roundtrip? migration partial-failure? CLI error paths? Windows-specific code)? Are the synthetic fixtures representative of real statement messiness? Is the coverage gate measuring the right package? Flakiness risks (network, keychain, tmpdir, timezone)? Does the local-real-statement layer have any CI story at all today (the self-hosted runner is decided-but-deferred)?
5. Check test hygiene: do any tests or fixtures embed realistic-looking sensitive data? Do snapshot tests write real parsed data into testdata/snapshots (and is that dir gitignored)?

Return the structured report: map = what the test/CI architecture actually is; issues = gaps/risks with evidence; recommendations split short-term vs long-term. Structured output only.

2. user (2026-06-10T16:13:33.371Z)

1 name: ci

```

2
3   on:
4     # All-branch pushes so feature branches are gated even before a PR exists
5     # (the regression fixtures in tests/test_regression_synth.py run here, secret-free).
6     push:
7     pull_request:
8     workflow_dispatch:
9
10  jobs:
11    test:
12      runs-on: ${{ matrix.os }}
13      strategy:
14        fail-fast: false
15      matrix:
16        os: [ubuntu-latest]
17        python-version: ["3.11", "3.12", "3.13"]
18        include:
19          # The dev box is Windows: test it on the primary Python (DPAPI secret backend,
Windows
20          # path handling, the sqlcipher3 win_amd64 wheel). Linux covers the 3.11/3.12
spread.
21          - os: windows-latest
22            python-version: "3.13"
23      steps:
24        - uses: actions/checkout@v6
25        - uses: actions/setup-python@v6
26        with:
27          python-version: ${{ matrix.python-version }}
28          cache: pip
29          cache-dependency-path: pyproject.toml
30        - name: Install
31          run: |
32            python -m pip install --upgrade pip
33            python -m pip install -e '.[dev]'
34        - name: Lint
35          run: ruff check .
36        - name: Format check
37          run: ruff format --check .
38        - name: Type check
39          run: mypy
40        - name: Test
41          run: pytest --cov=muneem --cov-report=term-missing --cov-fail-under=80
42
43    ocr:
44      # OCR tier needs the Tesseract binary; runs on demand (Linux) to avoid per-PR
flakiness.
45      if: github.event_name == 'workflow_dispatch'
46      runs-on: ubuntu-latest
47      steps:
48        - uses: actions/checkout@v6
49        - uses: actions/setup-python@v6
50        with:
51          python-version: "3.13"
52          cache: pip
53          cache-dependency-path: pyproject.toml
54        - name: Install Tesseract
55          run: sudo apt-get update && sudo apt-get install -y tesseract-ocr
56        - name: Install
57          run: |
58            python -m pip install --upgrade pip
59            python -m pip install -e '.[dev]'
60        - name: OCR tests
61          run: pytest -m ocr
62

```

3. user (2026-06-10T16:13:33.830Z)

```
1      # Convenience targets for Linux/macOS development.
2      # Windows users: use the pip/venv commands in README.md (or run under WSL).
3
4      PYTHON ?= python3
5      VENV    := .venv
6      BIN     := $(VENV)/bin
7
8      .PHONY: install lint format test test-ocr run clean hooks
9
10     # create venv + editable install (with dev deps)
11     $(BIN)/muneem: pyproject.toml
12         $(PYTHON) -m venv $(VENV)
13         $(BIN)/pip install --upgrade pip
14         $(BIN)/pip install -e ".[dev]"
15
16     install: $(BIN)/muneem hooks
17
18     # install the git hooks: pre-commit (blocks committing statements/secrets) and
19     # pre-push (runs lint + the offline test suite, incl. the DB invariant tripwires).
20     hooks:
21         git config core.hooksPath .githooks
22         chmod +x .githooks/pre-commit .githooks/pre-push
23
24     # run ruff (lint + format check)
25     lint: install
26         $(BIN)/ruff check .
27         $(BIN)/ruff format --check .
28
29     # auto-format with ruff (modifies files)
30     format: install
31         $(BIN)/ruff format .
32
33     # run the offline test suite
34     test: install
35         $(BIN)/pytest
36
37     # run OCR tests (needs the tesseract binary)
38     test-ocr: install
39         $(BIN)/pytest -m ocr
40
41     # show resolved local paths
42     run: install
43         $(BIN)/muneem info
44
45     # remove venv and caches
46     clean:
47         rm -rf $(VENV) .pytest_cache .ruff_cache src/muneem.egg-info
48
```

4. user (2026-06-10T16:13:34.431Z)

```
1      [build-system]
2      requires = ["setuptools>=61"]
3      build-backend = "setuptools.build_meta"
4
5      [project]
6      name = "muneem"
7      description = "A local-first personal-finance bookkeeper that ingests financial
documents from email into a consolidated ledger."
8      readme = "README.md"
9      requires-python = ">=3.11"
10     dynamic = ["version"]
11     # Private, single-user tool ? this classifier makes packaging tools refuse to upload it
```

```

to PyPI.
12     classifiers = ["Private :: Do Not Upload"]
13     dependencies = [
14         "typer>=0.12",
15         "platformdirs>=4.0",
16         "keyring>=25.0",
17         "pdfminer.six>=20240706",
18         "pypdfium2>=4.30",
19         "pytesseract>=0.3.10",
20         "pillow>=10.0",
21         "pypdf>=4.0",
22         "google-api-python-client>=2.100",
23         "google-auth>=2.30",
24         "google-auth-oauthlib>=1.2",
25         "pdfplumber>=0.11",
26         "sqlcipher3>=0.6",
27         "cryptography>=42",
28     ]
29
30     [project.scripts]
31     muneem = "muneem.cli:app"
32
33     [project.optional-dependencies]
34     outlook = ["msal>=1.0", "requests>=2.0"]
35     dev = [
36         "pytest>=8.0",
37         "pytest-cov>=5.0",
38         "ruff>=0.6",
39         "pyyaml>=6.0",
40         "mypy>=1.8",
41     ]
42
43     [tool.setuptools.dynamic]
44     version = { attr = "muneem.__version__" }
45
46     [tool.setuptools.packages.find]
47     where = ["src"]
48
49     [tool.ruff]
50     line-length = 100
51     src = ["src", "tests"]
52
53     [tool.ruff.lint]
54     select = ["E", "F", "I", "UP", "B"]
55
56     [tool.ruff.format]
57     quote-style = "double"
58     indent-style = "space"
59     skip-magic-trailing-comma = false
60     line-ending = "auto"
61
62     [tool.ruff.lint.flake8-bugbear]
63     # Typer uses Option()/Argument() as parameter defaults by design.
64     extend-immutable-calls = ["typer.Option", "typer.Argument"]
65
66     [tool.mypy]
67     files = ["src/muneem"]
68     python_version = "3.11"
69     ignore_missing_imports = true    # not all third-party libs ship type stubs
70     check_untyped_defs = true
71     no_implicit_optional = true
72     warn_redundant_casts = true
73     warn_unused_ignores = true
74     warn_unused_configs = true
75

```

```

76     # The email-ingestion providers (gmail/imap/outlook) are experimental scaffolding with
77     # NotImplemented methods (docs/CURRENT_STATE.md §1). Type-harden them when the `fetch`
command
78     # lands (ROADMAP B.4); until then don't gate CI on their half-built signatures.
79     [[tool.mypy.overrides]]
80     module = "muneem.ingestion.*"
81     ignore_errors = true
82
83     [tool.pytest.ini_options]
84     # Default runs are offline and deterministic: the ocr/llm/local_statements tiers
85     # (see docs/TESTING.md) are opt-in and excluded here.
86     addopts = "-ra -m 'not ocr and not llm and not local_statements'"
87     testpaths = ["tests"]
88     markers = [
89         "ocr: requires the Tesseract binary (image-OCR tier)",
90         "llm: makes live LLM calls (never in default CI)",
91         "local_statements: reads gitignored real statements (local only)",
92     ]
93

```

5. user (2026-06-10T16:13:34.832Z)

```

.githubhooks\pre-commit
.githubhooks\pre-push
.githubhooks\precommit_check.py

```

6. user (2026-06-10T16:13:39.873Z)

```

1     #!/bin/sh
2     # muneem pre-commit guard ? blocks staging of sensitive files (real statements,
3     # secrets, local DBs). See .githubhooks/precommit_check.py for the rules.
4     # Locates a Python interpreter so this works on Linux/WSL and Git-for-Windows.
5     # Enable with: git config core.hooksPath .githubhooks (or `make hooks`).
6     dir=$(dirname "$0")
7     for py in python3 python py; do
8         if command -v "$py" >/dev/null 2>&1; then
9             exec "$py" "$dir/precommit_check.py"
10        fi
11    done
12    echo "muneem pre-commit: no python interpreter found; skipping the sensitive-file
guard." >&2
13    echo " (install python to enable; tests/test_repo_hygiene.py is the post-commit
backstop)" >&2
14    exit 0
15

```

7. user (2026-06-10T16:13:40.349Z)

```

1     #!/bin/sh
2     # muneem pre-push gate ? runs lint + the offline test suite (including the DB invariant
3     # tripwires in tests/test_db_invariants.py) so only green code reaches GitHub. This is
the
4     # LOCAL substitute for server-side branch protection, which GitHub gates behind Pro /
public
5     # repos (this repo is private by design).
6     #
7     # Bypass intentionally (the "explicitly allowed" escape hatch) with:
8     #     git push --no-verify
9     #
10    # Enable with: git config core.hooksPath .githubhooks (or `make hooks`).
11
12    root=$(git rev-parse --show-toplevel 2>/dev/null) || exit 0
13
14    # Prefer the project venv (it has ruff + pytest); fall back to a PATH python.

```

```

15     if [ -x "$root/.venv/bin/python" ]; then
16         py="$root/.venv/bin/python"
17     elif [ -x "$root/.venv/Scripts/python.exe" ]; then
18         py="$root/.venv/Scripts/python.exe"
19     else
20         py=""
21         for cand in python3 python py; do
22             if command -v "$cand" >/dev/null 2>&1; then py="$cand"; break; fi
23         done
24     fi
25
26     if [ -z "$py" ]; then
27         echo "muneem pre-push: no python/venv found; skipping the lint+test gate (CI still
runs)." >&2
28         exit 0
29     fi
30
31     # If the dev tooling isn't installed in this interpreter, skip rather than false-block a
push.
32     if ! "$py" -m pytest --version >/dev/null 2>&1; then
33         echo "muneem pre-push: pytest not available in $py; skipping gate (run 'make
install')." >&2
34         echo "  CI still runs on push; or override now with: git push --no-verify" >&2
35         exit 0
36     fi
37
38     fail() {
39         echo "muneem pre-push: $1 ? push BLOCKED." >&2
40         echo "  Fix it, or bypass intentionally with: git push --no-verify" >&2
41         exit 1
42     }
43
44     echo "muneem pre-push: ruff check ..." >&2
45     "$py" -m ruff check "$root" || fail "ruff check failed"
46     echo "muneem pre-push: ruff format --check ..." >&2
47     "$py" -m ruff format --check "$root" || fail "ruff format check failed"
48     if "$py" -m mypy --version >/dev/null 2>&1; then
49         echo "muneem pre-push: mypy ..." >&2
50         (cd "$root" && "$py" -m mypy) || fail "mypy failed"
51     fi
52     echo "muneem pre-push: pytest (offline suite) ..." >&2
53     "$py" -m pytest -q "$root" || fail "tests failed"
54
55     echo "muneem pre-push: all checks passed." >&2
56     exit 0
57

```

8. user (2026-06-10T16:13:40.824Z)

```

1     #!/usr/bin/env python3
2     """muneem sensitive-file guard (invoked by the pre-commit hook).
3
4     Refuses to *stage* files that policy says must never be committed: real
5     statements (any PDF, or anything under ``testdata/`` outside the allowed
6     corpora), OAuth secrets, password rules, local databases, key material, and
7     ``.env`` files. This is the PRE-commit backstop to ``.gitignore`` ? it stops the
8     ``git add -f`` / ``git add -A`` mistake before it becomes a commit (the very
9     thing that caused an earlier statement leak). The post-commit complement is
10    ``tests/test_repo_hygiene.py``.
11
12    Exit 0 when nothing sensitive is staged, 1 otherwise. A deliberate, reviewed
13    exception can be committed with ``git commit --no-verify``.
14    """
15
16    from __future__ import annotations

```

```

17
18 import fnmatch
19 import os
20 import subprocess
21 import sys
22
23 # Only these subtrees of testdata/ are non-sensitive and may be committed.
24 ALLOWED_PREFIXES = ("testdata/promo-emails/", "testdata/synthetic/")
25 SECRET_NAMES = {"credentials.json", "token.json", "token.pickle", "passwords.txt"}
26
27
28 def is_sensitive(path: str) -> str | None:
29     """Return a human-readable reason if ``path`` must not be committed, else None."""
30     p = path.replace("\\", "/").strip()
31     if not p or p.startswith(ALLOWED_PREFIXES):
32         return None
33     name = p.rsplit("/", 1)[-1]
34     low = p.lower()
35     nlow = name.lower()
36     if p.startswith("testdata/"):
37         return "under testdata/ ? sensitive by default (only promo-emails/ and
synthetic/ allowed)"
38     if low.endswith(".pdf"):
39         return "PDF outside the allowed corpora (real statements must stay local)"
40     if low.endswith(("db", ".sqlite", ".sqlite3")):
41         return "local database (may contain parsed statement data)"
42     if low.endswith(".plaintext.bak"):
43         return "plaintext database backup (cleartext from 'muneem db encrypt'
migration)"
44     if fnmatch.fnmatch(nlow, "*recovery*.json"):
45         return "DB-key recovery file (passphrase-wrapped key ? keep it off the repo)"
46     if low.endswith(("pem", ".key")):
47         return "key material"
48     if nlow in SECRET_NAMES:
49         return "OAuth / credential secret"
50     if nlow.startswith("oauth_token"):
51         return "OAuth token"
52     # Block the rules *data* file (password_rules.yaml/.yaml/.json/?) but not the loader
module
53     # (src/muneem/password_rules.py is code, not a secret).
54     if fnmatch.fnmatch(nlow, "password_rules.*") and not nlow.endswith(".py"):
55         return "password rules"
56     if (nlow == ".env" or nlow.startswith(".env.")) and nlow != ".env.example":
57         return ".env file (may contain secrets)"
58     return None
59
60
61 def _staged_paths() -> list[str]:
62     # Tests inject a newline-separated list via PRECOMMIT_TEST_FILES (no git needed).
63     override = os.environ.get("PRECOMMIT_TEST_FILES")
64     if override is not None:
65         return [ln for ln in override.replace("\r\n", "\n").split("\n") if ln.strip()]
66     # -z keeps paths with spaces/unicode intact (the real statements have spaces).
67     out = subprocess.run(
68         ["git", "diff", "--cached", "--name-only", "-z", "--diff-filter=ACM"],
69         capture_output=True,
70         text=True,
71         check=True,
72     ).stdout
73     return [p for p in out.split("\0") if p.strip()]
74
75
76 def main() -> int:
77     offenders = [(p, reason) for p in _staged_paths() if (reason := is_sensitive(p))]
78     if not offenders:

```

```

79         return 0
80     print("muneem pre-commit: refusing to commit sensitive file(s):", file=sys.stderr)
81     for p, reason in offenders:
82         print(f" x {p}\n         {reason}", file=sys.stderr)
83     print(
84         "\nThese must never reach git (CLAUDE.md security rules; see .gitignore).\n"
85         "If this is a deliberate, reviewed exception: git commit --no-verify",
86         file=sys.stderr,
87     )
88     return 1
89
90
91 if __name__ == "__main__":
92     raise SystemExit(main())
93

```

9. user (2026-06-10T16:13:41.666Z)

```

tests\test_smoke.py
tests\test_db.py
tests\test_extract.py
tests\test_promo_regression.py
tests\test_ocr.py
tests\test_unlock.py
tests\ingestion\test_gmail.py
tests\ingestion\test_providers.py
tests\test_offers.py
tests\test_repo_hygiene.py
tests\parsers\test_engine.py
tests\parsers\test_hdfc.py
tests\parsers\test_registry.py
tests\parsers\test_validation.py
tests\parsers\test_axis.py
tests\parsers\test_clustering.py
tests\parsers\test_local_axis.py
tests\parsers\test_au.py
tests\parsers\test_local_au.py
tests\synthpdf.py
tests\test_regression_synth.py
tests\parsers\test_savings.py
tests\test_eval_cli.py
tests\conftest.py
tests\test_cli.py
tests\test_db_crypto.py
tests\parsers\test_parse_errors.py
tests\test_errors.py
tests\test_log.py
tests\test_redaction.py
tests\test_password_rules.py
tests\test_secrets.py
tests\test_statement_unlock.py
tests\test_db_crypto_ops.py
tests\test_verifier_properties.py
tests\test_dependency_licenses.py
tests\test_precommit_hook.py
tests\test_recovery.py
tests\test_db_invariants.py
tests\test_migrations.py
tests\test_provenance.py

```

10. user (2026-06-10T16:13:48.850Z)

```

1     import json
2     import os
3     from pathlib import Path

```



```

4
5     import pytest
6
7
8     @pytest.fixture(autouse=True)
9     def _isolate_keychain(monkeypatch):
10         """Never touch the real OS keychain in tests.
11
12         Replaces :mod:`muneem.secrets` with an in-memory store so the suite cannot pollute
the
13         developer's keychain (e.g. minting a DB key) and runs cleanly in headless CI where
no
14         keyring backend exists. Tests that need specific keychain behaviour re-patch on top.
15         """
16         store: dict[str, str] = {}
17
18         def _set(name, value):
19             store[name] = value
20
21         monkeypatch.setattr("muneem.secrets.get_secret", store.get)
22         monkeypatch.setattr("muneem.secrets.set_secret", _set)
23         monkeypatch.setattr("muneem.secrets.delete_secret", lambda name: store.pop(name,
None))
24         return store
25
26
27     @pytest.fixture
28     def snapshot():
29         def _assert_snapshot(name: str, data: list | dict):
30             snapshot_dir = Path("testdata/snapshots")
31             snapshot_dir.mkdir(parents=True, exist_ok=True)
32             snapshot_file = snapshot_dir / f"{name}.json"
33
34             if os.environ.get("UPDATE_SNAPSHOTS") == "1":
35                 with snapshot_file.open("w", encoding="utf-8") as f:
36                     json.dump(data, f, indent=2, ensure_ascii=False)
37                 return True
38
39             if not snapshot_file.exists():
40                 pytest.fail(f"Snapshot not found at {snapshot_file}. Run with
UPDATE_SNAPSHOTS=1.")
41
42             with snapshot_file.open("r", encoding="utf-8") as f:
43                 expected = json.load(f)
44
45             assert data == expected, f"Data does not match snapshot {name}"
46             return True
47
48         return _assert_snapshot
49

```

11. user (2026-06-10T16:13:49.316Z)

```

1     """Rugged DB invariants ? CI tripwires that block a PR when a *crucial* database
guarantee
2     regresses, **unless the change is made explicit in the same PR**.
3
4     Two kinds of guard:
5
6     1. **Security / integrity invariants** that must ALWAYS hold ? a failure is a real
regression to
7         fix, never something to "allow":
8         - the production ledger is *never* written in plaintext (encryption seam not
bypassed);
9         - a keyed open never silently downgrades to plaintext if the SQLCipher binding is

```

```

missing;
10         - foreign keys are enforced (and cascade);
11         - `documents.sha256` dedup is unique.
12
13     2. **Schema lock** ? pins ``SCHEMA_VERSION`` and the exact table/column shape. An
*accidental*
14     schema drift fails CI. A *deliberate* schema change is "explicitly allowed" by
updating
15     ``EXPECTED_SCHEMA`` / ``EXPECTED_SCHEMA_VERSION`` below **in the same PR** (a
visible, reviewable
16     diff) ? ideally paired with a migration. This is the tripwire the request asks for:
schema
17     churn can't sneak through a merge unnoticed.
18
19     All checks are offline, fast, and use fabricated data only.
20     ""
21
22     from __future__ import annotations
23
24     import sqlite3
25     import sys
26
27     import pytest
28
29     from muneem import db as dbmod
30
31     # --- Schema lock
-----
32     # Update BOTH of these (and add a migration) when you intentionally change the schema.
That edit
33     # is the "explicitly allowed" signal ? it shows up in the PR diff and forces a conscious
decision.
34
35     EXPECTED_SCHEMA_VERSION = 5 # baseline v4 + forward migration v5 (provenance)
36
37     # table -> [(column, declared_type, notnull, pk), ...] as reported by PRAGMA table_info.
38     EXPECTED_SCHEMA: dict[str, list[tuple[str, str, int, int]]] = {
39         "au_transactions": [
40             ("id", "INTEGER", 0, 1),
41             ("document_id", "INTEGER", 1, 0),
42             ("account_number", "TEXT", 0, 0),
43             ("txn_date", "TEXT", 0, 0),
44             ("value_date", "TEXT", 0, 0),
45             ("particulars", "TEXT", 0, 0),
46             ("debit", "REAL", 0, 0),
47             ("credit", "REAL", 0, 0),
48             ("balance", "REAL", 0, 0),
49         ],
50         "axis_cc_transactions": [
51             ("id", "INTEGER", 0, 1),
52             ("document_id", "INTEGER", 1, 0),
53             ("account_number", "TEXT", 0, 0),
54             ("txn_date", "TEXT", 0, 0),
55             ("particulars", "TEXT", 0, 0),
56             ("merchant_category", "TEXT", 0, 0),
57             ("amount", "REAL", 0, 0),
58             ("type", "TEXT", 0, 0),
59         ],
60         "axis_transactions": [
61             ("id", "INTEGER", 0, 1),
62             ("document_id", "INTEGER", 1, 0),
63             ("account_number", "TEXT", 0, 0),
64             ("txn_date", "TEXT", 0, 0),
65             ("value_date", "TEXT", 0, 0),
66             ("particulars", "TEXT", 0, 0),

```

```

67         ("debit", "REAL", 0, 0),
68         ("credit", "REAL", 0, 0),
69         ("balance", "REAL", 0, 0),
70     ],
71     "documents": [
72         ("id", "INTEGER", 0, 1),
73         ("source", "TEXT", 1, 0),
74         ("external_id", "TEXT", 0, 0),
75         ("sender", "TEXT", 0, 0),
76         ("subject", "TEXT", 0, 0),
77         ("received_date", "TEXT", 0, 0),
78         ("doc_type", "TEXT", 0, 0),
79         ("sha256", "TEXT", 1, 0),
80         ("path", "TEXT", 0, 0),
81         ("status", "TEXT", 1, 0),
82         ("created_at", "TEXT", 1, 0),
83     ],
84     "hdfc_transactions": [
85         ("id", "INTEGER", 0, 1),
86         ("document_id", "INTEGER", 1, 0),
87         ("account_number", "TEXT", 0, 0),
88         ("txn_date", "TEXT", 0, 0),
89         ("value_date", "TEXT", 0, 0),
90         ("particulars", "TEXT", 0, 0),
91         ("ref", "TEXT", 0, 0),
92         ("withdrawal", "REAL", 0, 0),
93         ("deposit", "REAL", 0, 0),
94         ("closing_balance", "REAL", 0, 0),
95     ],
96     "offers": [
97         ("id", "INTEGER", 0, 1),
98         ("document_id", "INTEGER", 1, 0),
99         ("merchant", "TEXT", 0, 0),
100        ("promo_code", "TEXT", 0, 0),
101        ("description", "TEXT", 0, 0),
102        ("expiry_raw", "TEXT", 0, 0),
103        ("expiry_type", "TEXT", 0, 0),
104        ("source", "TEXT", 1, 0),
105        ("redeemed", "INTEGER", 1, 0),
106        ("source_text", "TEXT", 0, 0),
107    ],
108    # Provenance/confidence per document ? first-class but a SEPARATE table (DESIGN
$8.7), added by
109    # forward migration v5. Txn rows reach it via their documents(id) FK; not duplicated
per row.
110    "provenance": [
111        ("id", "INTEGER", 0, 1),
112        ("document_id", "INTEGER", 1, 0),
113        ("source_type", "TEXT", 1, 0),
114        ("parser", "TEXT", 0, 0),
115        ("parser_version", "TEXT", 0, 0),
116        ("mapping_layer", "TEXT", 0, 0),
117        ("confidence", "TEXT", 0, 0),
118        ("created_at", "TEXT", 1, 0),
119    ],
120    # Bookkeeping for the hand-rolled migration runner (muneem/migrations.py). Present
from the
121    # baseline; not a data table, but pinned here so its shape can't drift silently
either.
122    "schema_migrations": [
123        ("version", "INTEGER", 0, 1),
124        ("name", "TEXT", 1, 0),
125        ("checksum", "TEXT", 1, 0),
126        ("applied_at", "TEXT", 1, 0),
127    ],

```

```

128     }
129
130
131     def _schema_fingerprint(conn: sqlite3.Connection) -> dict[str, list[tuple[str, str, int,
int]]]:
132         tables = [
133             r[0]
134             for r in conn.execute(
135                 "SELECT name FROM sqlite_master WHERE type='table' AND name NOT LIKE
'sqlite_%' "
136                 "ORDER BY name"
137             )
138         ]
139         out: dict[str, list[tuple[str, str, int, int]]] = {}
140         for t in tables:
141             cols = conn.execute(f"PRAGMA table_info({t})").fetchall()
142             # (cid, name, type, notnull, dflt_value, pk) -> (name, TYPE, notnull, pk)
143             out[t] = [(c[1], (c[2] or "").upper(), int(c[3]), int(c[5])) for c in cols]
144         return out
145
146
147     def test_schema_version_is_locked():
148         with dbmod.connect(":memory:") as conn:
149             dbmod.init_db(conn)
150             actual = conn.execute("PRAGMA user_version").fetchone()[0]
151             assert actual == EXPECTED_SCHEMA_VERSION, (
152                 f"SCHEMA_VERSION changed ({actual} != {EXPECTED_SCHEMA_VERSION}). If
intentional, update "
153                 "EXPECTED_SCHEMA_VERSION here + add a migration (the explicit allow)."
154             )
155
156
157     def test_schema_shape_is_locked():
158         with dbmod.connect(":memory:") as conn:
159             dbmod.init_db(conn)
160             actual = _schema_fingerprint(conn)
161             if actual != EXPECTED_SCHEMA:
162                 diffs: list[str] = []
163                 for t in sorted(set(EXPECTED_SCHEMA) | set(actual)):
164                     if t not in actual:
165                         diffs.append(f" - table REMOVED: {t}")
166                     elif t not in EXPECTED_SCHEMA:
167                         diffs.append(f" - table ADDED: {t} -> {actual[t]}")
168                     elif actual[t] != EXPECTED_SCHEMA[t]:
169                         diffs.append(
170                             f" - table CHANGED: {t}\n          expected {EXPECTED_SCHEMA[t]}\n"
171                             f"          actual   {actual[t]}"
172                         )
173                 raise AssertionError(
174                     "DB schema lock tripped ? the on-disk shape changed:\n"
175                     + "\n".join(diffs)
176                     + "\n\nIf this change is INTENTIONAL: update EXPECTED_SCHEMA (and bump "
177                     + "EXPECTED_SCHEMA_VERSION + add a migration) in this same PR. That edit is
the "
178                     + "explicit allow. If it is NOT intentional, you just caught a regression."
179                 )
180
181
182     # --- Security / integrity invariants (must always hold)
183     -----
184
185     def test_foreign_keys_are_enforced():
186         with dbmod.connect(":memory:") as conn:
187             assert conn.execute("PRAGMA foreign_keys").fetchone()[0] == 1

```

```

188
189
190 def test_document_delete_cascades_to_offers():
191     with dbmod.connect(":memory:") as conn:
192         dbmod.init_db(conn)
193         doc = dbmod.insert_document(conn, source="file", sha256="s1", status="parsed")
194         dbmod.insert_offer(conn, document_id=doc, promo_code="X")
195         assert len(dbmod.list_offers(conn)) == 1
196         conn.execute("DELETE FROM documents WHERE id = ?", (doc,))
197         assert dbmod.list_offers(conn) == [] # cascade removed the orphan
198
199
200 def test_documents_sha256_is_unique():
201     with dbmod.connect(":memory:") as conn:
202         dbmod.init_db(conn)
203         dbmod.insert_document(conn, source="file", sha256="dup", status="parsed")
204         with pytest.raises(sqlite3.IntegrityError):
205             dbmod.insert_document(conn, source="file", sha256="dup", status="parsed")
206
207
208 def test_cli_ingest_writes_an_encrypted_ledger(tmp_path):
209     """End-to-end guard: the production CLI must write an ENCRYPTED ledger. Catches
anyone
210     swapping the encryption seam (connect_encrypted / _ledger) back to a plaintext
open."""
211     pytest.importorskip("sqlcipher3")
212     from typer.testing import CliRunner
213
214     from muneem.cli import app
215     from synthpdf import at, build_pdf
216
217     pdf = tmp_path / "promo.pdf"
218     build_pdf([at(72, 72, "Use code SAVE20 at checkout")], pdf)
219     led = tmp_path / "ledger.db"
220
221     result = CliRunner().invoke(app, ["ingest", str(pdf), "--db", str(led)])
222     assert result.exit_code == 0, result.output
223     assert led.exists(), "ingest did not create the ledger"
224     assert not dbmod.is_plaintext_sqlite(led), (
225         "production ingest wrote a PLAINTEXT ledger ? the encryption seam was bypassed.
"
226         "Real financial data must never land in cleartext."
227     )
228
229
230 def test_keyed_open_never_silently_downgrades_to_plaintext(tmp_path, monkeypatch):
231     """If the SQLCipher binding is unavailable, a keyed open must RAISE ? never quietly
write a
232     plaintext database in its place."""
233     p = tmp_path / "ledger.db"
234     monkeypatch.setitem(sys.modules, "sqlcipher3", None) # makes `import sqlcipher3`
raise
235     with pytest.raises(dbmod.EncryptedDatabaseError):
236         with dbmod.connect(p, key="00" * 32):
237             pass
238     assert not dbmod.is_plaintext_sqlite(p), "a plaintext DB must NOT be created as a
fallback"
239

```

12. user (2026-06-10T16:13:49.627Z)

```

1     """Security guard: no sensitive paths may be tracked in git.
2
3     Encodes DESIGN.md section 5 ("Nothing sensitive ever in git") and docs/TESTING.md
4     section 9 as a failing-build assertion. If this test fails, something that must

```

```

5     stay local has been committed.
6     """
7
8     from __future__ import annotations
9
10    import subprocess
11    from pathlib import Path
12
13    REPO_ROOT = Path(__file__).resolve().parents[1]
14
15    FORBIDDEN_PREFIXES = ("testdata/statements/", "testdata/private/")
16    FORBIDDEN_SUFFIXES = (".db", ".sqlite", ".sqlite3", ".env", ".pem", ".key")
17    FORBIDDEN_NAMES = (
18        "token.json",
19        "token.pickle",
20        "credentials.json",
21        "password_rules.yaml",
22        "password_rules.yml",
23        "password_rules.json",
24        "passwords.txt",
25    )
26
27
28    def _tracked_files() -> list[str]:
29        out = subprocess.run(
30            ["git", "ls-files"],
31            cwd=REPO_ROOT,
32            capture_output=True,
33            text=True,
34            check=True,
35        )
36        return [line.strip() for line in out.stdout.splitlines() if line.strip()]
37
38
39    def test_no_sensitive_paths_tracked():
40        offenders = []
41        for path in _tracked_files():
42            low = path.lower()
43            if (
44                low.startswith(FORBIDDEN_PREFIXES)
45                or low.endswith(FORBIDDEN_SUFFIXES)
46                or Path(path).name.lower() in FORBIDDEN_NAMES
47            ):
48                offenders.append(path)
49        assert not offenders, f"sensitive paths must never be tracked: {offenders}"
50
51
52    def test_only_corpus_pdfs_are_tracked():
53        """Statements are PDFs ? the only PDFs git may track are the non-sensitive promo
54        corpus.
55
56        This is the guard that would have caught real HDFC statements being committed to
57        testdata/hdfc-statements/.
58        """
59        offenders = [
60            path
61            for path in _tracked_files()
62            if path.lower().endswith(".pdf") and not path.startswith("testdata/promo-
63            emails/")
64        ]
65        assert not offenders, f"only testdata/promo-emails/ PDFs may be tracked; found:
66        {offenders}"
67
68    def test_no_noncorpus_testdata_is_tracked():

```

```

67     """Mirror of the pre-commit guard: under testdata/, only the non-sensitive
68     promo-emails/ and synthetic/ corpora may be tracked ? nothing else (real
69     statements, zips, credentials, snapshots all stay local)."""
70     allowed = ("testdata/promo-emails/", "testdata/synthetic/")
71     offenders = [
72         path
73         for path in _tracked_files()
74         if path.startswith("testdata/") and not path.startswith(allowed)
75     ]
76     assert not offenders, (
77         f"only promo-emails/ and synthetic/ may be tracked under testdata/; found:
{offenders}"
78     )
79

```

13. user (2026-06-10T16:13:54.164Z)

```

1     """Layer-1 regression CI ? full-pipeline fixtures on synthetic born-digital PDFs.
2
3     Generates *real* PDFs (no committed statements, no secrets) and runs the production
pipeline ?
4     ``extract`` (pdfminer) ? ``get_parser`` ? ``parse_and_insert`` ? assert a **reconciled**
parse. This
5     covers the extraction / coordinate-clustering / engine layers end-to-end, the gap the
synthetic
6     word-list & table unit tests (which bypass pdfplumber) don't reach. See ROADMAP §
"Planned ?
7     Regression CI for statement parsing" (Layer 1). Always-on, secret-free ? runs in cloud
CI.
8     """
9
10    from pathlib import Path
11
12    from muneem import db as dbmod
13    from muneem.extract import extract
14    from muneem.parsers import get_parser
15    from synthpdf import at, build_pdf, right
16
17
18    def _hdfc_pdf(path: Path) -> Path:
19        """HDFC layout: leading date x0?52, narration x0?111, three right-aligned amount
columns whose
20        right edges land in the clusterer's bands (withdrawal <400, deposit <490, balance
?490), plus a
21        SUMMARY block for the bookend. Three rows reconcile (opening 10,000 ? closing
10,000)."""
22        p = [
23            at(50, 60, "HDFC BANK"),
24            at(50, 72, "Savings Account Details"),
25            at(50, 84, "Account Number : 50100012345678"),
26            # header ? must carry "Narration" + a "Closing" token so _table_body locates the
table
27            at(72, 110, "Date"),
28            at(175, 110, "Narration"),
29            at(300, 110, "Withdrawals"),
30            at(390, 110, "Deposits"),
31            at(480, 110, "Closing"),
32            at(515, 110, "Balance"),
33        ]
34        rows = [
35            ("05/02/2026", "UPI-SWIGGY-PAY", "500.00", "0.00", "9,500.00"),
36            ("08/02/2026", "SALARY-CREDIT", "0.00", "2,000.00", "11,500.00"),
37            ("10/02/2026", "ATM-WDL", "1,500.00", "0.00", "10,000.00"),
38        ]
39        top = 130

```

```

40     for d, narr, wd, dep, bal in rows:
41         p += [
42             at(52, top, d),
43             at(111, top, narr),
44             right(358, top, wd),
45             right(443, top, dep),
46             right(561, top, bal),
47         ]
48         top += 15
49     p += [
50         at(50, 200, "SUMMARY"),
51         at(50, 212, "Opening Balance Debit Amount Credit Amount Closing Balance"),
52         at(50, 224, "10,000.00 2,000.00 2,000.00 10,000.00"),
53         at(50, 236, "Debit Count Credit Count"),
54         at(50, 248, "2 1"),
55     ]
56     return build_pdf(p, path)
57
58
59 def _au_savings_pdf(path: Path) -> Path:
60     """Savings layout (the shared savings.py engine path used by Axis/AU): a clean
61     date | narration | debit | credit | balance table in well-separated columns ?
62     clustering ? engine mapping ? per-row reconcile. Three rows that walk (opening
63     50,000)."""
64     p = [
65         at(50, 60, "AU Small Finance Bank"),
66         at(50, 72, "Account Number : 1234567890123"),
67         # header (clustering's date+money filter drops it; here only for realism / text
68         markers)
69         at(52, 100, "Date"),
70         at(110, 100, "Transaction"),
71         at(310, 100, "Withdrawal"),
72         at(395, 100, "Deposit"),
73         at(485, 100, "Balance"),
74     ]
75     rows = [
76         ("02/08/2025", "UPI-ZOMATO", "1,200.00", "", "48,800.00"),
77         ("05/08/2025", "SALARY", "", "30,000.00", "78,800.00"),
78         ("10/08/2025", "ATM-CASH", "2,000.00", "", "76,800.00"),
79     ]
80     top = 124
81     for d, narr, debit, credit, bal in rows:
82         line = [at(52, top, d), at(110, top, narr), right(520, top, bal)]
83         if debit:
84             line.append(right(340, top, debit))
85         if credit:
86             line.append(right(420, top, credit))
87         p += line
88         top += 15
89     return build_pdf(p, path)
90
91 def _run_pipeline(pdf: Path) -> tuple[str, int, str]:
92     """Mirror `muneem ingest`: extract text ? pick parser ? parse+insert ? read back
93     status."""
94     ex = extract(pdf)
95     parser = get_parser(ex.text)
96     assert parser is not None, "no parser routed this synthetic statement"
97     with dbmod.connect(":memory:") as conn:
98         dbmod.init_db(conn)
99         doc_id = dbmod.insert_document(conn, source="file", sha256=ex.sha256,
100 status="parsed")
101     n = parser.parse_and_insert(pdf, conn, doc_id)
102     status = dbmod.get_document_by_sha(conn, ex.sha256)["status"]

```



```

100         return type(parser).__name__, n, status
101
102
103     def test_hdfc_layout_full_pipeline_reconciles(tmp_path):
104         name, n, status = _run_pipeline(_hdfc_pdf(tmp_path / "hdfc.pdf"))
105         assert name == "HDFCParser"
106         assert n == 3, f"expected 3 transactions, got {n}"
107         assert status == "reconciled", f"expected reconciled, got {status}"
108
109
110     def test_savings_layout_full_pipeline_reconciles(tmp_path):
111         name, n, status = _run_pipeline(_au_savings_pdf(tmp_path / "au.pdf"))
112         assert name == "AUParser"
113         assert n == 3, f"expected 3 transactions, got {n}"
114         assert status == "reconciled", f"expected reconciled, got {status}"
115

```

14. user (2026-06-10T16:13:54.597Z)

```

1     """Dependency-free, born-digital PDF generator for regression fixtures (stdlib only).
2
3     Layer 1 of the regression-CI plan (ROADMAP § "Planned ? Regression CI") needs full-
pipeline
4     fixtures ? real PDFs that go through pdfminer/pdfplumber, not synthetic word-lists that
bypass them.
5     Rather than add a dependency (reportlab etc.), we emit a minimal valid PDF with
positioned
6     Helvetica text** (`Tm`/`Tj`), so it stays commercial-safe and CI-portable.
7
8     Coordinates: callers think in pdfplumber's top-down `top` (distance from the page top);
we convert
9     to PDF's bottom-up baseline. Absolute `top` need not be exact ? parsers key on
x-positions, same-row
10    grouping, and row order ? but words on one row share a baseline (one `top`) and rows are
separated.
11    Amount columns are right-aligned via real Helvetica metrics so their right edges
(`x1`) land in
12    the bands the HDFC clusterer / engine expect.
13    """
14
15    from __future__ import annotations
16
17    from dataclasses import dataclass
18    from pathlib import Path
19
20    # Helvetica advance widths (per 1000 em). Exact for the chars that matter for right-
aligned amounts
21    # (digits / comma / period / minus / space); a reasonable default covers labels (left-
aligned, so
22    # their width is irrelevant to placement).
23    _W = {
24        " ": 278,
25        "!": 278,
26        "'": 355,
27        "#": 556,
28        "$": 556,
29        "%": 889,
30        "&": 667,
31        "'": 191,
32        "(": 333,
33        ")": 333,
34        "*": 389,
35        "+": 584,
36        ",": 278,
37        "-": 333,

```

```

38     ".": 278,
39     "/" : 278,
40     "0": 556,
41     "1": 556,
42     "2": 556,
43     "3": 556,
44     "4": 556,
45     "5": 556,
46     "6": 556,
47     "7": 556,
48     "8": 556,
49     "9": 556,
50     ":" : 278,
51     "," : 278,
52     "<": 584,
53     "=": 584,
54     ">": 584,
55     "?": 556,
56     "@": 1015,
57 }
58 _UPPER = 667 # rough average for A-Z
59 _LOWER = 528 # rough average for a-z
60
61
62 def helvetica_width(text: str, size: float) -> float:
63     """Advance width of ``text`` in Helvetica at ``size`` points."""
64     total = 0
65     for ch in text:
66         if ch in _W:
67             total += _W[ch]
68         elif ch.isupper():
69             total += _UPPER
70         else:
71             total += _LOWER
72     return total / 1000.0 * size
73
74
75 @dataclass
76 class Placement:
77     """One text run: its LEFT edge ``x``, top-down ``top``, and the string."""
78
79     x: float
80     top: float
81     text: str
82     size: float = 9.0
83
84
85 def at(x: float, top: float, text: str, size: float = 9.0) -> Placement:
86     """Left-aligned run starting at ``x``."""
87     return Placement(x=x, top=top, text=text, size=size)
88
89
90 def right(x1: float, top: float, text: str, size: float = 9.0) -> Placement:
91     """Right-aligned run whose right edge lands at ``x1`` (for amount columns)."""
92     return Placement(x=x1 - helvetica_width(text, size), top=top, text=text, size=size)
93
94
95 def _esc(text: str) -> str:
96     return text.replace("\\", r"\\").replace("(", r"\(").replace(")", r"\)")
97
98
99 def build_pdf(
100     placements: list[Placement], path: Path | str, *, width: float = 595.0, height:
float = 842.0
101 ) -> Path:

```

```

102     """Write a single-page PDF with the given positioned text. Returns the path.
103
104     Hand-rolls a minimal PDF (catalog ? pages ? page ? Helvetica font ? content stream)
with a
105     byte-accurate xref table so pdfminer/pdfplumber parse it.
106     """
107     ops: list[str] = []
108     for p in placements:
109         y = height - p.top # top-down -> PDF bottom-up baseline
110         ops.append(f"BT /F1 {p.size:.2f} Tf 1 0 0 1 {p.x:.2f} {y:.2f} Tm
({_esc(p.text))} Tj ET")
111     content = ("\n".join(ops)).encode("latin-1")
112
113     objects: list[bytes] = [
114         b"<< /Type /Catalog /Pages 2 0 R >>",
115         b"<< /Type /Pages /Kids [3 0 R] /Count 1 >>",
116         (
117             f"<< /Type /Page /Parent 2 0 R /MediaBox [0 0 {width:.0f} {height:.0f}] "
118             f"/Resources << /Font << /F1 4 0 R >> >> /Contents 5 0 R >>"
119         ).encode("latin-1"),
120         b"<< /Type /Font /Subtype /Type1 /BaseFont /Helvetica >>",
121         b"<< /Length " + str(len(content)).encode() + b" >>\nstream\n" + content +
b"\nendstream",
122     ]
123
124     out = bytearray(b"%PDF-1.4\n")
125     offsets: list[int] = []
126     for i, obj in enumerate(objects, start=1):
127         offsets.append(len(out))
128         out += f"{i} 0 obj\n".encode() + obj + b"\nendobj\n"
129
130     xref_pos = len(out)
131     count = len(objects) + 1
132     out += f"xref\n0 {count}\n".encode()
133     out += b"0000000000 65535 f \n"
134     for off in offsets:
135         out += f"{off:010d} 00000 n \n".encode()
136     out += (
137         b"trailer\n<< /Size " + str(count).encode() + b" /Root 1 0 R >>\n"
138         b"startxref\n" + str(xref_pos).encode() + b"\n%%EOF"
139     )
140
141     out_path = Path(path)
142     out_path.write_bytes(out)
143     return out_path
144
145
146     __all__ = ["Placement", "at", "right", "helvetica_width", "build_pdf"]
147

```

15. user (2026-06-10T16:14:01.710Z)

```

1     """Property-style fuzzing of the reconciliation verifier ? deterministic (seeded), no
new dep.
2
3     The verifier is muneem's truth oracle: an off-by-one in the balance walk or the total
identity
4     would silently let mislabeled rows through. These tests generate many random
*consistent* ledgers
5     and assert the oracles accept them, then perturb a single value past the tolerance and
assert the
6     oracles catch it. A one-line slip in the reconcile arithmetic fails here.
7     """
8
9     import random

```

```

10
11 import pytest
12
13 from muneem.parsers.concepts import ConceptRow
14 from muneem.parsers.validation import reconcile_running_balance, reconcile_total
15
16
17 def _consistent_ledger(rng: random.Random, n: int, opening: float):
18     """Build a random clean ledger of ``n`` rows; return (rows, closing_balance)."""
19     rows: list[ConceptRow] = []
20     bal = opening
21     for _ in range(n):
22         amt = round(rng.uniform(0.01, 5000.0), 2)
23         if rng.random() < 0.5:
24             bal = round(bal - amt, 2)
25             rows.append(ConceptRow(txn_date="01/01/2026", narration="x", debit=amt,
balance=bal))
26         else:
27             bal = round(bal + amt, 2)
28             rows.append(ConceptRow(txn_date="01/01/2026", narration="x", credit=amt,
balance=bal))
29     return rows, bal
30
31
32 def _entries(rows):
33     return [(r.debit, r.credit, r.balance) for r in rows]
34
35
36 @pytest.mark.parametrize("seed", range(60))
37 def test_consistent_ledger_always_reconciles(seed):
38     rng = random.Random(seed)
39     opening = round(rng.uniform(0, 100_000), 2)
40     rows, closing = _consistent_ledger(rng, rng.randint(2, 40), opening)
41     assert reconcile_running_balance(_entries(rows)).ok
42     assert reconcile_total(rows, opening=opening, closing=closing)
43
44
45 @pytest.mark.parametrize("seed", range(60))
46 def test_single_balance_perturbation_is_caught_by_the_walk(seed):
47     rng = random.Random(10_000 + seed)
48     opening = round(rng.uniform(0, 100_000), 2)
49     rows, closing = _consistent_ledger(rng, rng.randint(2, 40), opening)
50     i = rng.randrange(len(rows))
51     r = rows[i]
52     rows[i] = ConceptRow( # corrupt one balance by more than the tolerance
53         txn_date=r.txn_date,
54         narration=r.narration,
55         debit=r.debit,
56         credit=r.credit,
57         balance=round((r.balance or 0.0) + 1.0, 2),
58     )
59     assert not reconcile_running_balance(_entries(rows)).ok
60
61
62 @pytest.mark.parametrize("seed", range(60))
63 def test_total_oracle_catches_a_wrong_closing_bookend(seed):
64     rng = random.Random(20_000 + seed)
65     opening = round(rng.uniform(0, 100_000), 2)
66     rows, closing = _consistent_ledger(rng, rng.randint(2, 40), opening)
67     assert reconcile_total(rows, opening=opening, closing=closing)
68     assert not reconcile_total(rows, opening=opening, closing=round(closing + 1.0, 2))
69

```

```

1      """Local-only checks for the Axis savings parser against real encrypted statements.
2
3      Skipped unless an Axis password is available (`password_rules.yaml` or
4      `MUNEEM_TEST_AXIS_PW`); never runs in CI. The savings parser is **oracle-gated**
5      (coordinate clustering + `extract_tables` ? engine mapping ? reconciliation), so this
6      asserts the safety invariant: it either stored nothing (a vintage it can't yet read) or
7      everything it stored reconciled ? never mislabeled rows. Surfaces counts / booleans /
8      parser names only ? no statement content.
9      """
10
11     import os
12     from pathlib import Path
13
14     import pdfplumber
15     import pytest
16
17     from muneem import db as dbmod
18     from muneem.parsers import get_parser
19     from muneem.password_rules import get_password_for_bank
20
21     AXIS_PDFS = sorted(Path("testdata/axis-statements").glob("*.pdf"))
22
23
24     @pytest.mark.local_statements
25     @pytest.mark.parametrize("pdf_path", AXIS_PDFS, ids=lambda p: p.name)
26     def test_local_axis_never_stores_unreconciled(pdf_path, tmp_path):
27         pw = get_password_for_bank("axis") or os.environ.get("MUNEEM_TEST_AXIS_PW")
28         if not pw:
29             pytest.skip("needs the Axis password (password_rules.yaml or
MUNEEM_TEST_AXIS_PW)")
30
31         with pdfplumber.open(pdf_path, password=pw) as pdf:
32             text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
len(pdf.pages))))
33             parser = get_parser(text)
34             if parser is None:
35                 pytest.skip(f"{pdf_path.name}: no parser recognizes it yet (Axis rewrite
pending)")
36
37             with dbmod.connect(tmp_path / "t.db") as conn:
38                 dbmod.init_db(conn)
39                 doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
status="parsed")
40                 n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
41                 status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
42                 stored = dbmod.list_axis_transactions(conn)
43
44             # Oracle-gated invariant: the savings parser either stores nothing (and flags the
45             # document unreconciled) or, if it stored rows, they reconciled. Never garbage.
46             if type(parser).__name__ == "AxisParser":
47                 assert (n == 0 and stored == []) or status == "reconciled", (
48                     f"{pdf_path.name}: stored {n} row(s) with status={status} ? "
49                     "stop-gap must not persist unreconciled rows"
50                 )
51

```

17. user (2026-06-10T16:14:02.041Z)

```

1      """Local-only checks for the AU Bank parser against real encrypted statements.
2
3      Skipped unless an AU password is available (`password_rules.yaml` or
`MUNEEM_TEST_AU_PW`); never
4      runs in CI. The parser is **oracle-gated**, so this asserts the safety invariant: it
either stored
5      nothing (a vintage it can't yet read) or everything it stored reconciled ? never

```

```

mislabeled rows.
6     Surfaces counts / booleans / parser names only ? no statement content.
7     """
8
9     import os
10    from pathlib import Path
11
12    import pdfplumber
13    import pytest
14
15    from muneem import db as dbmod
16    from muneem.parsers import get_parser
17    from muneem.password_rules import get_password_for_bank
18
19    AU_PDFS = sorted(Path("testdata/au-bank-statements").glob("*.pdf"))
20
21
22    @pytest.mark.local_statements
23    @pytest.mark.parametrize("pdf_path", AU_PDFS, ids=lambda p: p.name)
24    def test_local_au_never_stores_unreconciled(pdf_path, tmp_path):
25        pw = (
26            get_password_for_bank("aubank")
27            or get_password_for_bank("au")
28            or os.environ.get("MUNEEM_TEST_AU_PW")
29        )
30        if not pw:
31            pytest.skip("needs the AU password (password_rules.yaml or MUNEEM_TEST_AU_PW)")
32
33        with pdfplumber.open(pdf_path, password=pw) as pdf:
34            text = "\n".join((pdf.pages[i].extract_text() or "") for i in range(min(2,
35len(pdf.pages))))
36            parser = get_parser(text)
37            if parser is None or type(parser).__name__ != "AUParser":
38                pytest.skip(f"{pdf_path.name}: not routed to AUParser")
39
40            with dbmod.connect(tmp_path / "t.db") as conn:
41                dbmod.init_db(conn)
42                doc_id = dbmod.insert_document(conn, source="file", sha256=pdf_path.name,
43status="parsed")
44                n = parser.parse_and_insert(pdf_path, conn, doc_id, password=pw)
45                status = dbmod.get_document_by_sha(conn, pdf_path.name)["status"]
46                stored = dbmod.list_au_transactions(conn)
47
48                assert (n == 0 and stored == []) or status == "reconciled", (
49                    f"{pdf_path.name}: stored {n} row(s) with status={status} ? must not persist
unreconciled"
50                )

```

18. user (2026-06-10T16:14:07.994Z)

```

1     """Regression over the committed promo corpus, driven by expected.yaml.
2
3     Covers the deterministic tiers only: T1 (text-layer codes must be recovered) and
4     T3 (no-code emails must yield nothing). T2 (image-only, needs OCR) and T4
5     (unverified) are intentionally excluded here ? see docs/TESTING.md sections 3-4.
6     """
7
8     from pathlib import Path
9
10    import pytest
11    import yaml
12
13    from muneem.extract import extract
14    from muneem.offers import extract_offer

```

```

15
16 CORPUS = Path(__file__).resolve().parents[1] / "testdata" / "promo-emails"
17 ANSWERS = yaml.safe_load((CORPUS / "expected.yaml").read_text(encoding="utf-8"))
18
19 DETERMINISTIC = sorted(
20     name
21     for name, spec in ANSWERS.items()
22     if spec.get("verified") and spec.get("tier") in {"T1", "T3"}
23 )
24
25
26 @pytest.mark.parametrize("name", DETERMINISTIC)
27 def test_promo_codes_match_answer_key(name):
28     spec = ANSWERS[name]
29     got = extract_offer(extract(CORPUS / name).text).codes
30     if spec["code_location"] == "none":
31         assert got == [], f"{name}: expected no codes, got {got}"
32     else:
33         for code in spec["codes"]:
34             assert code in got, f"{name}: expected {code!r} in {got}"
35

```

19. user (2026-06-10T16:14:08.255Z)

```

1     """Enforce muneem's commercial-license principle (DESIGN.md § 5 rule 8).
2
3     Every declared dependency must be on the permissive allowlist below (kept in sync
4     with docs/DEPENDENCIES.md). Adding a dependency without recording its
5     MIT/BSD/Apache/? license here fails the build, forcing a license review ? and
6     keeps banned copyleft licenses (notably AGPL, e.g. PyMuPDF) out of the project.
7     """
8
9     from __future__ import annotations
10
11     import re
12     import tomllib
13     from pathlib import Path
14
15     REPO_ROOT = Path(__file__).resolve().parents[1]
16
17     # Normalized (lowercased) dependency name -> license. Permissive / commercial-safe only.
18     ALLOWLIST = {
19         "typer": "MIT",
20         "platformdirs": "MIT",
21         "keyring": "MIT",
22         "pdfminer.six": "MIT",
23         "pypdfium2": "Apache-2.0/BSD-3-Clause",
24         "pytesseract": "Apache-2.0",
25         "pillow": "HPND",
26         "pypdf": "BSD-3-Clause",
27         "google-api-python-client": "Apache-2.0",
28         "google-auth": "Apache-2.0",
29         "google-auth-oauthlib": "Apache-2.0",
30         "pdfplumber": "MIT",
31         # SQLCipher at-rest encryption. Binding (sqlcipher3) = Zlib; bundled native deps are
32         # SQLCipher core = BSD-3-Clause, SQLite = Public Domain, OpenSSL = Apache-2.0 ? all
33         # permissive (see docs/DEPENDENCIES.md).
34         "sqlcipher3": "Zlib",
35         # PyCA cryptography ? Argon2id + AES-GCM for the passphrase-wrapped recovery file.
36         "cryptography": "Apache-2.0 OR BSD-3-Clause",
37         "pytest": "MIT",
38         "pytest-cov": "MIT",
39         "ruff": "MIT",
40         "pyyaml": "MIT",
41         "mypy": "MIT",

```

```

42     "msal": "MIT",
43     "requests": "Apache-2.0",
44 }
45
46 BANNED_SUBSTRINGS = ("GPL", "AGPL", "AFFERO")
47
48
49 def _declared_dependency_names() -> set[str]:
50     data = tomlib.loads((REPO_ROOT / "pyproject.toml").read_text(encoding="utf-8"))
51     specs = list(data["project"].get("dependencies", []))
52     for extra in data["project"].get("optional-dependencies", {}).values():
53         specs.extend(extra)
54     names = set()
55     for spec in specs:
56         # "pdfminer.six>=20240706" / "typer[all]>=0.12" -> bare name
57         name = re.split(r"[<=>!~;\[ ]", spec, maxsplit=1)[0].strip().lower()
58         if name:
59             names.add(name)
60     return names
61
62
63 def test_every_dependency_is_allowlisted():
64     unknown = sorted(_declared_dependency_names() - set(ALLOWLIST))
65     assert not unknown, (
66         f"dependencies not on the commercial-safe allowlist: {unknown}. "
67         "Add them to docs/DEPENDENCIES.md and ALLOWLIST with their license, or remove
68     them."
69 )
70
71 def test_allowlist_contains_no_banned_licenses():
72     bad = {
73         name: lic
74         for name, lic in ALLOWLIST.items()
75         if any(token in lic.upper() for token in BANNED_SUBSTRINGS)
76     }
77     assert not bad, f"copyleft licenses are banned but present in allowlist: {bad}"
78

```

20. user (2026-06-10T16:14:14.997Z)

```

tests\conftest.py:28: def snapshot():
tests\conftest.py:29:     def _assert_snapshot(name: str, data: list | dict):
tests\conftest.py:30:         snapshot_dir = Path("testdata/snapshots")
tests\conftest.py:31:         snapshot_dir.mkdir(parents=True, exist_ok=True)
tests\conftest.py:32:         snapshot_file = snapshot_dir / f"{name}.json"
tests\conftest.py:35:         with snapshot_file.open("w", encoding="utf-8") as f:
tests\conftest.py:39:             if not snapshot_file.exists():
tests\conftest.py:40:                 pytest.fail(f"Snapshot not found at {snapshot_file}. Run with
UPDATE_SNAPSHOTS=1.")
tests\conftest.py:42:             with snapshot_file.open("r", encoding="utf-8") as f:
tests\conftest.py:45:                 assert data == expected, f"Data does not match snapshot {name}"
tests\conftest.py:48:     return _assert_snapshot
tests\test_repo_hygiene.py:69:     statements, zips, credentials, snapshots all stay local)."""

```

21. user (2026-06-10T16:14:15.316Z)

```

1     # --- Sensitive data ? NEVER commit ---
2     # Real financial documents stay local only. Everything under testdata/ is sensitive by
3     # default ? ONLY the non-sensitive promo corpus and committed synthetic fixtures are
4     # Real statements in ANY subdir (e.g. testdata/hdfc-statements/) must never be
5     committed.
6     testdata/*
7     !testdata/promo-emails/

```



```
7      !testdata/synthetic/
8      *.db
9      *.sqlite
10     *.sqlite3
11     muneem.db*
12
13     # Secrets and credentials
14     .env
15     .env.*
16     !.env.example
17     *.pem
18     *.key
19     credentials.json
20     token.json
21     token.pickle
22     oauth_token*
23     password_rules.yaml
24     password_rules.yml
25     password_rules.json
26     passwords.txt
27
28     # Logs (may contain extracted statement text ? sanitize before committing any log)
29     *.log
30     logs/
31
32     # --- Python ---
33     __pycache__/
34     *.py[cod]
35     *$py.class
36     *.so
37     .Python
38     build/
39     develop-eggs/
40     dist/
41     downloads/
42     eggs/
43     .eggs/
44     lib/
45     lib64/
46     parts/
47     sdist/
48     var/
49     wheels/
50     *.egg-info/
51     .installed.cfg
52     *.egg
53     MANIFEST
54     .venv/
55     venv/
56     env/
57     ENV/
58     .pytest_cache/
59     .mypy_cache/
60     .ruff_cache/
61     .coverage
62     htmlcov/
63     .tox/
64
65     # --- Go ---
66     *.exe
67     *.exe~
68     *.dll
69     *.dylib
70     *.test
71     *.out
```

```
72     vendor/
73
74     # --- Node (if a UI later lands) ---
75     node_modules/
76     npm-debug.log*
77     yarn-debug.log*
78     yarn-error.log*
79     .next/
80     .nuxt/
81     dist/
82
83     # --- IDE / OS ---
84     .vscode/
85     .idea/
86     *.swp
87     *.swo
88     *~
89     .DS_Store
90     Thumbs.db
91     desktop.ini
92
93     # --- Jupyter (in case notebooks are used for exploration) ---
94     .ipynb_checkpoints/
95
96     # --- Scratch / local-only ---
97     scratch/
98     local/
99     tmp/
100    ruff-bin/
101    ruff.zip
102
103
```

22. user (2026-06-10T16:14:25.977Z)

```
testdata/promo-emails/README.md
testdata/promo-emails/airasia-1.pdf
testdata/promo-emails/airasia-2.pdf
testdata/promo-emails/airasia-3.pdf
testdata/promo-emails/dataquest.pdf
testdata/promo-emails/expected.yaml
testdata/promo-emails/flipkart.pdf
testdata/promo-emails/makemytrip-1.pdf
testdata/promo-emails/makemytrip-2.pdf
testdata/promo-emails/swiggy.pdf
testdata/promo-emails/tataneu.pdf
testdata/promo-emails/unacademy.pdf
---untracked sensitive dirs---
aubank-statements
axis-statements
credentials
hdfc-statements
promo-emails
snapshots
```

23. user (2026-06-10T16:14:27.674Z)

```
---
d5f4442 Lay the design foundation for muneem
```

24. user (2026-06-10T16:14:36.008Z)

```
testdata/credentials/:
total 8
```

```
drwxr-xr-x 1 avidu 197609 0 May 31 00:33 .
drwxr-xr-x 1 avidu 197609 0 May 31 08:37 ..
-rw-r--r-- 1 avidu 197609 413 May 31 00:33
client_secret_207319377548-j0vbfmqak4cq6kqa4s9ikq6l5sgore.apps.googleusercontent.com.json
```

```
testdata/snapshots/:
total 16
drwxr-xr-x 1 avidu 197609 0 May 31 08:37 .
drwxr-xr-x 1 avidu 197609 0 May 31 08:37 ..
-rw-r--r-- 1 avidu 197609 2525 May 31 04:11 April Credit Card Statement.json
-rw-r--r-- 1 avidu 197609 6074 May 31 04:11 lder Credit Card Statement.json
---counts---
7
7
4
```

25. user (2026-06-10T16:14:45.615Z)

```
testdata/snapshots\April Credit Card Statement.json -> list of 12 items; item keys:
['account_number', 'amount', 'merchant_category', 'particulars', 'txn_date', 'type']
testdata/snapshots\lder Credit Card Statement.json -> list of 28 items; item keys:
['account_number', 'amount', 'merchant_category', 'particulars', 'txn_date', 'type']
```

26. user (2026-06-10T16:15:00.460Z)

```
=== tests/test_smoke.py ===
from muneem import __version__

def test_version_is_a_nonempty_string():
    assert isinstance(__version__, str)
    assert __version__
=== tests/test_db.py ===
from muneem import db as dbmod

def test_schema_and_offer_roundtrip():
    with dbmod.connect(":memory:") as conn:
        dbmod.init_db(conn)
        doc_id = dbmod.insert_document(
            conn, source="file", external_id="x.pdf", sha256="abc123", status="parsed"
        )
        dbmod.insert_offer(conn, document_id=doc_id, promo_code="SAVE10")
        rows = dbmod.list_offers(conn)
        assert len(rows) == 1
=== tests/test_extract.py ===
from muneem.extract import _normalize

def test_normalize_folds_ff_ligature():
    # U+FB00 is the '?' ligature PyMuPDF returns for words like "Offer" in the corpus.
    assert _normalize("O?er ends") == "Offer ends"

def test_normalize_is_idempotent_on_ascii():
    assert _normalize("Use code: JS231sz") == "Use code: JS231sz"
=== tests/test_ocr.py ===
"""OCR tier (T2) regression ? needs the Tesseract binary, so it is marked `ocr`
and excluded from the default suite (docs/TESTING.md § 3). Matching is fuzzy
because OCR of stylized codes is approximate (§ 6).
"""

import difflib
from pathlib import Path
```

```

import pytest
import yaml

from muneem.ocr import ocr_pdf, tesseract_available
=== tests/test_unlock.py ===
import os
from pathlib import Path

import pypdf
import pytest

from muneem import unlock

def _write_pdf(path: Path, password: str | None = None) -> Path:
    writer = pypdf.PdfWriter()
    writer.add_blank_page(width=200, height=200)
=== tests/test_offers.py ===
from muneem.offers import extract_offer

def test_code_after_use_code_cue():
    assert extract_offer("Last chance! Use code: JS231sz today").codes == ["JS231sz"]

def test_code_after_using_code_with_extra_space():
    # Mirrors makemytrip-2's "Using code:  EARLYBIRDDEAL" (double space).
    assert extract_offer("Book now. Using code:  EARLYBIRDDEAL").codes == ["EARLYBIRDDEAL"]

=== tests/test_eval_cli.py ===
"""Tests for `muneem eval` ? the seed?golden onboarding harness.

Runs the CLI against a folder of *synthetic* statements (no secrets, no real data) and checks
the
reconcile-rate report. Reuses the born-digital fixtures from the regression suite.
"""

from typer.testing import CliRunner

from muneem.cli import app
from test_regression_synth import _au_savings_pdf, _hdfc_pdf

runner = CliRunner()
=== tests/test_cli.py ===
from pathlib import Path

from typer.testing import CliRunner

from muneem.cli import app
from muneem.extract import Extracted

runner = CliRunner()

def test_version_flag():
    result = runner.invoke(app, ["--version"])
=== tests/test_db_crypto.py ===
"""At-rest encryption for the local ledger (SQLCipher via sqlcipher3).

Covers the safety-critical invariants of the encryption seam:
- a keyed open round-trips data through SQLCipher with SQL queries intact;
- a wrong key is rejected (never silently returns garbage / empty);
- the on-disk file is genuinely encrypted (stdlib sqlite3 cannot read it);
- ``:memory:`` and keyless opens stay plaintext (the test / CI path);

```

- the keychain-backed key is minted once and reused;
- a pre-existing *plaintext* ledger is refused by ``connect\_encrypted`` and can be migrated in place by ``encrypt\_database``;
- the key never appears in an error message.

=== tests/test_errors.py ===

"""The muneem exception hierarchy + that db re-exports the canonical EncryptedDatabaseError."""

from muneem import db as dbmod

from muneem.errors import EncryptedDatabaseError, MuneemError, ParseError, UnlockError

def test_all_errors_derive_from_muneem_error():

assert issubclass(ParseError, MuneemError)

assert issubclass(UnlockError, MuneemError)

assert issubclass(EncryptedDatabaseError, MuneemError)

=== tests/test_log.py ===

"""The redaction-aware logger scrubs sensitive substrings as a backstop."""

import logging

from muneem.log import RedactingFilter, get_logger

def test_redacting_filter_masks_sensitive_record(caplog):

logger = get_logger("muneem.test.redact")

logger.setLevel(logging.INFO)

with caplog.at_level(logging.INFO, logger="muneem.test.redact"):

logger.info("stored balance 1,23,456.78 for acct 998877665544")

=== tests/test_redaction.py ===

"""Redaction helpers keep sensitive data out of logs / exceptions / stdout."""

from pathlib import Path

from muneem.redaction import mask_secret, redact_filename, redact_text

def test_redact_filename_drops_path_and_masks_pii_digits():

out = redact_filename(Path("/home/avi/statements/Avi_Dullu_28022026.pdf"))

assert "/" not in out and "\\" not in out # no directory components

assert "statements" not in out and "home" not in out

assert "28022026" not in out # DOB-like digit run masked

=== tests/test_password_rules.py ===

"""password_rules loader: candidate collection, quote-numeric, config-dir precedence, legacy CWD warning, and malformed-YAML handling that never leaks the file's (password) contents.

No real password material ? fabricated values written to a temp config dir / cwd.

"""

import warnings

import pytest

from muneem import password_rules as pr

from muneem.errors import ConfigError

=== tests/test_secrets.py ===

"""The OS-keychain wrappers: graceful read, attributed (value-free) write, swallowed delete.

The autouse conftest fixture swaps ``muneem.secrets`` for an in-memory store, so we capture the *real* wrapper functions at import time and drive them by patching the underlying ``keyring`` lib.

"""

```

import keyring
import pytest
from keyring.errors import KeyringError

import muneem.secrets as secretsmod
from muneem.errors import KeychainError
=== tests/test_statement_unlock.py ===
"""Tests for the ingest-path password resolver (muneem.statement_unlock).

Synthetic encrypted PDFs (pypdf), no real statements. The OS keychain is replaced with an in-
memory
dict so tests never touch the real keyring (and run cleanly in headless CI). No real password.
"""

import json
from pathlib import Path

import pypdf
import pytest

=== tests/test_db_crypto_ops.py ===
"""Crypto operations on the encrypted ledger: key rotation, encrypted backup, migration temp
cleanup, correctness at size, and the plaintext-refusal guidance ? plus their CLI commands.

Offline, fabricated data only; the autouse conf test fixture keeps the real keychain untouched.
"""

import sqlite3

import pytest

pytest.importorskip("sqlcipher3")

=== tests/test_precommit_hook.py ===
"""Tests for the pre-commit sensitive-file guard (.github/hooks/precommit_check.py).

Tests the pure rule function + the exit-code path (via PRECOMMIT_TEST_FILES), so
no shell, git mutation, or real staging is needed ? portable across OSes.
"""

from __future__ import annotations

import importlib.util
from pathlib import Path

import pytest
=== tests/test_recovery.py ===
"""The passphrase-wrapped recovery file ? round-trip, wrong-passphrase / corruption rejection,
no-secret-leak, and the CLI setup-recovery ? (lost keychain) ? recover flow. Fabricated keys
only.
"""

import pytest
from typer.testing import CliRunner

from muneem import recovery
from muneem.recovery import RecoveryError

runner = CliRunner()

=== tests/test_migrations.py ===
"""The hand-rolled migration runner: baseline apply, idempotency, legacy back-fill, forward
migrations, baseline-drift detection, and running inside an encrypted connection.
"""

```

```

import pytest

from muneem import db as dbmod
from muneem import migrations
from muneem.errors import MigrationError

_BASE = dbmod._SCHEMA
_VER = dbmod.SCHEMA_VERSION # 4
=== tests/test_provenance.py ===
"""Provenance/confidence recording ? a first-class but SEPARATE table (DESIGN §8.7).

One provenance row per document (not per txn row), reached via the existing documents FK; upsert
on
re-parse; and the parsers record it (source/parser/version/confidence). Fabricated data only.
"""

from pathlib import Path

from muneem import db as dbmod
from muneem.parsers import au, axis
from muneem.parsers.concepts import ConceptRow
from muneem.parsers.savings import SavingsParse
=== tests/ingestion/test_gmail.py ===
"""Unit tests for the Gmail provider using a hand-rolled fake service."""

import base64

from muneem.ingestion.base import SearchCriteria
from muneem.ingestion.gmail import GmailProvider

def _b64(text: str) -> str:
    return base64.urlsafe_b64encode(text.encode()).decode()

=== tests/ingestion/test_providers.py ===
from muneem.ingestion.base import SearchCriteria
from muneem.ingestion.gmail import GmailProvider
from muneem.ingestion.imap import IMAPProvider
from muneem.ingestion.outlook import OutlookProvider

def test_gmail_query_translation():
    criteria = SearchCriteria(sender="alerts@hdfcbank.net", has_attachment=True,
newer_than_days=90)
    provider = GmailProvider("fake_creds.json")
    query = provider._build_query(criteria)

    assert "from:alerts@hdfcbank.net" in query
=== tests/parsers/test_engine.py ===
"""Tests for the generic concept-mapping engine (muneem.parsers.engine).

Exercises the ladder on synthetic cell-matrices (no PDF/DB): L0 header lexicon, the profile-
order
fallback, L1/L2 content search with the oracle picking the self-identifying balance column, and
a
deliberately non-bank ``FamilySpec`` to prove the engine is family-agnostic.
"""

import re
from dataclasses import dataclass

from muneem.parsers.engine import BANK_FAMILY, FamilySpec, Profile, map_table
from muneem.parsers.validation import parse_amount, reconcile_running_balance
=== tests/parsers/test_hdfc.py ===

```

```
"""Tests for the HDFC statement parser (muneem.parsers.hdfc).
```

```
Column logic is exercised on *synthetic* word lists that mimic the real statement geometry ? no real data, fast, offline. ``parse_and_insert`` is tested by monkeypatching the parse functions (not by mocking pdfplumber), and a ``local_statements`` test runs the real parser on the user's PDFs (skipped in CI).
"""
```

```
import os
from pathlib import Path
```

```
import pytest
=== tests/parsers/test_registry.py ===
"""Regression test for the parser registry (the PR #9-11 production-path bug).
```

```
`from muneem.parsers import get_parser` must return a working selector ? i.e. the registry must be populated by package import alone, without anything importing the axis/hdfc/... submodules first. We check in a FRESH interpreter (subprocess) so a prior test importing a submodule can't mask a regression.
"""
```

```
from __future__ import annotations
```

```
import subprocess
import sys
=== tests/parsers/test_validation.py ===
"""Tests for the verifier (muneem.parsers.validation): the per-row walk, the layout-independent total reconcile, the concept-level bookend, candidate selection, and the confidence verdict."""
```

```
from muneem.parsers.concepts import ConceptRow, StatementConcepts
from muneem.parsers.validation import (
    Confidence,
    reconcile_running_balance,
    reconcile_total,
    select_reconciling,
    verify,
)
```

```
=== tests/parsers/test_axis.py ===
"""Tests for the Axis parsers (savings via the concept core + credit card).
```

```
Savings runs end-to-end on synthetic data through a pdfplumber mock (extract_tables / extract_text / extract_words), so the per-table ? engine ? reconcile path is exercised without a real PDF; ``parse_and_insert``'s store-iff-reconciled gate is checked by monkeypatching that function. Real encrypted statements run under the ``local_statements`` marker (test_local_axis.py).
"""
```

```
from pathlib import Path
from unittest.mock import MagicMock, patch
```

```
import muneem.db as dbmod
=== tests/parsers/test_clustering.py ===
"""Tests for coordinate clustering (muneem.parsers.clustering).
```

```
Synthetic word geometry only ? proves lines group by ``top``, columns come from the transaction band (a full-width header must not collapse them), and a single wide cell can't merge two columns.
"""
```

```
from muneem.parsers.clustering import cluster_to_matrix, group_lines, infer_columns
```

```
def W(text, x0, top, x1=None):
    """A minimal pdfplumber-style word dict."""
```



```

    return {
=== tests/parsers/test_au.py ===
"""Tests for the AU Bank parser.

AU runs on the shared engine-driven savings path (covered in depth by test_axis / test_engine),
so
here we check AU detection, an end-to-end per-row reconcile via a pdfplumber mock, and the
store-iff-reconciled gate. Real statements run under local_statements (test_local_au.py).
"""

from pathlib import Path
from unittest.mock import MagicMock, patch

import muneem.db as dbmod
from muneem.parsers import au
=== tests/parsers/test_savings.py ===
"""Tests for the shared savings path's balance-delta reconstruction (muneem.parsers.savings).

`_reconstruct_by_balance_delta` recovers debit/credit from the running-balance delta when
columns
don't split cleanly (a merged amount column, an interleaved value-date column, sparse deposits).
It
is exercised here on synthetic cell-matrices ? no PDF, no secrets ? including the *dropped-row*
case
(a balance jump matching no single amount) which must NOT reconcile (the real Axis May 2025
case).
"""

from muneem.parsers import savings
from muneem.parsers.profiles import AXIS_PROFILE
from muneem.parsers.validation import reconcile_running_balance

=== tests/parsers/test_parse_errors.py ===
"""Every parser wraps an unexpected failure in a clean, non-leaking ``ParseError``.

A PDF library can raise with statement text in its message; the muneem-level error must expose
only
a redacted filename (no full path, no statement contents, no PII digits), while preserving the
original cause via exception chaining for local debugging.
"""

from pathlib import Path

import pytest

from muneem import db as dbmod

```

27. assistant (2026-06-10T16:15:12.609Z)

Now let me check docs/TESTING.md and run the offline suite.

28. user (2026-06-10T16:15:13.759Z)

```

CURRENT_STATE.md
DEPENDENCIES.md
ROADMAP.md
TESTING.md
encryption-at-rest.md
feedbackJune2026.md
ingesting-statements.md
onboarding-a-bank.md
parser-architecture.md
prior-art-email-mining.md
research-open-banking-india.md
schema.md

```

muneem ? Testing Strategy

> Status: ****active.**** Companion to [``ROADMAP.md``](./ROADMAP.md) (sequencing) and [``./DESIGN.md``](./DESIGN.md) (architecture + threat model). This doc owns **how we know muneem works** ? and, just as importantly, **how we know it isn't leaking or hallucinating**.

>

> Created 2026-05-30. Grounded in a direct inspection of the committed promo corpus (``testdata/promo-emails/``), not in the abstract.

1. Principles

1. ****Grounded, not aspirational.**** Every test tier below maps to a failure mode actually observed in the real corpus (§ 2), not a hypothetical one.
 2. ****Precision ranks with recall.**** A bookkeeper that invents a promo code or a transaction is worse than one that misses it. Negative fixtures (no code present) are first-class.
 3. ****Test on safe data by default.**** The committed promo corpus is non-sensitive and drives CI. Sensitive paths (statements) are tested on ****synthetic/redacted fixtures**** in CI and on ****local-only, gitignored**** real data behind an opt-in marker ? never real statements in CI or the repo.
 4. ****CI is offline, deterministic, secretless.**** No network, no cloud LLM, no OAuth, no keychain in the default CI run. Non-deterministic steps (OCR, LLM) are mocked/cassette-backed by default and exercised live only behind explicit markers.
 5. ****Security posture is enforced by tests, not just docs.**** Redaction, category-gating of cloud LLMs, and "no sensitive paths tracked in git" are assertions that fail the build (§ 9).
 6. ****Adding a corpus file means adding its answer-key entry.**** The corpus and its ground truth move together (§ 4).
-

2. What the corpus actually contains (grounded findings)

Direct inspection of all 10 PDFs ? PyMuPDF text-layer extraction (``get_text``) + large-image count (``get_images``, 2300x300 px, matching the Email-Mining dedup filter) + visual rendering of a sample. This is the empirical basis for everything below.

PDF	Issuer (from content)	Pages	Text chars	Big imgs	Promo code	Code location	Expiry / validity	Verified
<code>`dataquest.pdf`</code>	Dataquest	3	2526	0	<code>`JS231sz`</code>	**text layer**	"Ends Today" + live countdown (relative to send date 2024-07-29)	? text + visual
<code>`makemytrip-2.pdf`</code>	MakeMyTrip	3	925	7	<code>`EARLYBIRDDEAL`</code>	**text layer**	"?15 days in advance" (conditional, not a date)	? text
<code>`swiggy.pdf`</code>	Swiggy × ClearTax	4	774	3	<code>`CTSWIGGYUX`</code>	**image only** (OCR)	"before July 31st" (in image)	? visual
<code>`flipkart.pdf`</code>	Flipkart	2	555	1	? none ?	n/a	"GOAT SALE, 20?25 Jul" (in image)	? negative
<code>`airasia-2.pdf`</code>	AirAsia	2	695	1	TBD	image?	"Flat 50% Off Seats, 25% Off Meals" (subject/text)	?? unverified
<code>`airasia-3.pdf`</code>	AirAsia	2	710	1	TBD	image?	"Final Day! Save 15% Off Flights?" (relative)	?? unverified
<code>`airasia-1.pdf`</code>	**Air India Express** (? filename)	3	834	4	TBD	image?	"Special Offers"	?? unverified
<code>`makemytrip-1.pdf`</code>	MakeMyTrip	2	1051	2	TBD	image?	"travel dates & save up?"	?? unverified
<code>`unacademy.pdf`</code>	Unacademy	2	555	3	TBD	image?	"Last few hours left Ayush!" (relative, personalized)	?? unverified
<code>`tataneu.pdf`</code>	Tata Neu	4	1581	10	TBD	image?	?	?? unverified

****`Verified`**** distinguishes ground truth we have **confirmed** (text layer and/or visual read) from entries still ****TBD****. We do ****not**** invent codes for the unverified rows ? establishing them (via OCR + a human glance) is an explicit task in ROADMAP A.4. Attribution discipline (CLAUDE.md): if it isn't verified, it's marked unverified.

Gotchas the real data exposes (each becomes a test)

- **Ligatures in the text layer.** PyMuPDF returns `0?er` / `0?` (U+FB00 `?`), not `Offer`/`Off`. Naïve substring/regex matching for "Off"/"Offer" silently fails. ? **NFKC normalization** before matching, with a dedicated unit test (`airasia-2`, `airasia-3`).
- **Codes live in images about half the time.** 6 of 10 PDFs carry the offer in rasterized banners (`bigimgs` ? 1 and an empty/near-empty text layer). OCR is not optional for the corpus; it is the path for ~half of it.
- **Expiry is rarely a clean date.** Observed forms: absolute-ish ("before July 31st"), relative ("Ends Today", "Final Day!", "Last few hours left"), conditional ("?15 days in advance"), and a **live countdown timer**. ? Expiry parsing must resolve relative phrases against the email's **received date** and tolerate "no hard date."
- **Filename ? issuer.** `airasia-1.pdf` is an **Air India Express** mailer. ? Issuer detection must derive from sender/content, never the filename. (Also: fix the mislabel in the corpus README.)
- **Third-party PII in fixtures.** Recipient is `Ayush Raina <ayushraina767@gmail.com>` (the prototype's tester, *not* the muneem user), and bodies are personalized ("?left Ayush!"). The corpus README says "no PII beyond the recipient's email address"; in reality the recipient is a *different person* and appears in the body. Treated as **non-sensitive** (promo, already committed, no financial data), but extraction tests must not surface or log it as financial PII, and golden files (§ 5) intentionally retain it as committed non-sensitive content.

3. Fixture tiers (derived from § 2)

The corpus self-stratifies into capability tiers. Tests are gated by tier so CI stays green and honest about what's proven.

Tier	Capability under test	Members	CI behavior
---	---	---	---
T1	text-layer codes	Deterministic extraction, no OCR	`dataquest`, `makemytrip-2` Must pass in default CI (recall + precision = 100%).
T2	image-only codes	OCR (Tesseract) ? code recovery	`swiggy` (+ any unverified row confirmed image-coded) Behind `@pytest.mark.ocr`; runs in the optional OCR job, skipped if the Tesseract binary is absent.
T3	no-code negatives	Precision / no hallucination	`flipkart` (+ confirmed no-code rows) Must pass in default CI (assert <i>*zero*</i> codes extracted).
T4	unverified	Answer key not yet established	the 6 ?? rows `@pytest.mark.xfail(strict=False)` until verified in A.4; never block CI on guesses.

4. The answer-key file (ground truth)

A single committed file is the source of truth; tests read it, they don't hardcode expectations.

`testdata/promo-emails/expected.yaml` ? one entry per PDF:

```
```yaml
```

**Seed (created during ROADMAP A.4). verified:false rows are placeholders, NOT assertions**

```
dataquest.pdf:
 issuer: Dataquest
 codes: ["JS231sz"]
 code_location: text # text | image | none
 discount: "$706 / lifetime"
 expiry: {type: relative, raw: "Ends Today", resolved_against: received_date}
 verified: true
makemytrip-2.pdf:
 issuer: MakeMyTrip
 codes: ["EARLYBIRDDEAL"]
 code_location: text
```

29. user (2026-06-10T16:15:22.798Z)

tests\ingestion\test_providers.py ...	[ 1%]
tests\parsers\test_au.py ....	[ 2%]
tests\parsers\test_axis.py .....	[ 5%]
tests\parsers\test_clustering.py ....	[ 6%]
tests\parsers\test_engine.py .....	[ 7%]
tests\parsers\test_hdfc.py .....	[ 12%]
tests\parsers\test_parse_errors.py ....	[ 13%]
tests\parsers\test_registry.py ..	[ 13%]
tests\parsers\test_savings.py ....	[ 14%]
tests\parsers\test_validation.py .....	[ 17%]
tests\test_cli.py .....	[ 18%]
tests\test_db.py ..	[ 19%]
tests\test_db_crypto.py .....	[ 21%]
tests\test_db_crypto_ops.py .....	[ 24%]
tests\test_db_invariants.py .....	[ 26%]
tests\test_dependency_licenses.py ..	[ 26%]
tests\test_errors.py ..	[ 27%]
tests\test_eval_cli.py ..	[ 27%]
tests\test_extract.py ..	[ 28%]
tests\test_log.py ..	[ 28%]
tests\test_migrations.py .....	[ 30%]
tests\test_offers.py .....	[ 32%]
tests\test_password_rules.py .....	[ 34%]
tests\test_precommit_hook.py .....	[ 42%]
tests\test_promo_regression.py ...	[ 43%]
tests\test_provenance.py .....	[ 44%]
tests\test_recovery.py .....	[ 46%]
tests\test_redaction.py ....	[ 47%]
tests\test_regression_synth.py ..	[ 48%]
tests\test_repo_hygiene.py ...	[ 48%]
tests\test_secrets.py ....	[ 50%]
tests\test_smoke.py .	[ 50%]
tests\test_statement_unlock.py .....	[ 52%]
tests\test_unlock.py .....	[ 53%]
tests\test_verifier_properties.py .....	[ 63%]
.....	[ 82%]
.....	[100%]

===== tests coverage =====  
\_\_\_\_\_ coverage: platform win32, python 3.13.13-final-0 \_\_\_\_\_

Name	Stmts	Miss	Cover	Missing
-----				
src\muneem\__init__.py	1	0	100%	
src\muneem\cli.py	301	90	70%	46-48, 116-117, 123-124, 159-161, 165-169, 171, 182-186, 258-270, 276-277, 307-325, 335-361, 397-400, 402-405, 423-424, 443-446, 462-464, 486-487, 495-497, 528-530, 536
src\muneem\config.py	29	4	86%	42-43, 49-50
src\muneem\db.py	149	7	95%	130, 133-134, 222, 250, 418, 422
src\muneem\db_key.py	17	0	100%	
src\muneem\errors.py	8	0	100%	
src\muneem\extract.py	23	0	100%	
src\muneem\ingestion\__init__.py	6	0	100%	
src\muneem\ingestion\base.py	15	0	100%	
src\muneem\ingestion\gmail.py	82	20	76%	20-39, 53, 73, 104
src\muneem\ingestion\imap.py	42	21	50%	29-38, 49-61, 64, 70, 76
src\muneem\ingestion\outlook.py	66	41	38%	28-58, 72-96, 99, 105, 111
src\muneem\log.py	21	2	90%	25-26
src\muneem\migrations.py	46	0	100%	
src\muneem\ocr.py	35	15	57%	38, 41, 45, 54-68
src\muneem\offers.py	29	0	100%	
src\muneem\parsers\__init__.py	20	1	95%	22
src\muneem\parsers\au.py	35	1	97%	46

src\muneem\parsers\axis.py	95	16	83%	37-41, 77, 154-161, 175, 181
src\muneem\parsers\base.py	19	8	58%	16-21, 28, 35
src\muneem\parsers\clustering.py	65	3	95%	54, 86, 124
src\muneem\parsers\concepts.py	24	0	100%	
src\muneem\parsers\engine.py	179	6	97%	240-241, 322, 348, 357, 359
src\muneem\parsers\hdfc.py	212	16	92%	123-124, 193, 258, 314, 326, 350, 372-401, 432
src\muneem\parsers\profiles.py	7	0	100%	
src\muneem\parsers\savings.py	111	6	95%	119, 125, 137, 153, 162, 212
src\muneem\parsers\validation.py	85	2	98%	50-51
src\muneem\password_rules.py	41	1	98%	53
src\muneem\recovery.py	60	2	97%	88-89
src\muneem\redaction.py	19	0	100%	
src\muneem\secrets.py	20	0	100%	
src\muneem\statement_unlock.py	42	4	90%	39-40, 55-56
src\muneem\unlock.py	56	5	91%	34, 37-38, 76, 86

-----

TOTAL	1960	271	86%
-------	------	-----	-----

Required test coverage of 80% reached. Total coverage: 86.17%

===== 390 passed, 21 deselected in 6.35s =====

### 30. user (2026-06-10T16:16:09.601Z)

```

1 """The hand-rolled migration runner: baseline apply, idempotency, legacy back-fill,
forward
2 migrations, baseline-drift detection, and running inside an encrypted connection.
3 """
4
5 import pytest
6
7 from muneem import db as dbmod
8 from muneem import migrations
9 from muneem.errors import MigrationError
10
11 _BASE = dbmod._SCHEMA
12 _VER = dbmod.SCHEMA_VERSION # 4
13
14
15 def _applied(conn):
16 return {r[0]: r[1] for r in conn.execute("SELECT version, name FROM
schema_migrations")}
17
18
19 def test_fresh_db_applies_baseline():
20 with dbmod.connect(":memory:") as conn:
21 # Isolate baseline behaviour from the real forward migrations (migrations=[]).
22 v = migrations.run_migrations(
23 conn, baseline_sql=_BASE, baseline_version=_VER, migrations=[]
24)
25 assert v == _VER
26 assert conn.execute("PRAGMA user_version").fetchone()[0] == _VER
27 assert _VER in _applied(conn) and _applied(conn)[_VER] == "baseline"
28 names = {r[0] for r in conn.execute("SELECT name FROM sqlite_master WHERE
type='table'")}
29 assert {"documents", "offers", "schema_migrations"} <= names
30
31
32 def test_default_path_applies_provenance_v5():
33 # init_db uses the real MIGRATIONS list ? baseline (4) + provenance (5).
34 with dbmod.connect(":memory:") as conn:
35 dbmod.init_db(conn)
36 assert conn.execute("PRAGMA user_version").fetchone()[0] == 5
37 applied = _applied(conn)
38 assert applied.get(5) == "provenance"
39 names = {r[0] for r in conn.execute("SELECT name FROM sqlite_master WHERE

```

```

type='table'}}
40 assert "provenance" in names
41
42
43 def test_idempotent_rerun_changes_nothing():
44 with dbmod.connect(":memory:") as conn:
45 migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
46 before = _applied(conn)
47 migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
48 assert _applied(conn) == before
49
50
51 def test_legacy_db_is_backfilled_without_re_running_ddl():
52 with dbmod.connect(":memory:") as conn:
53 # Pre-framework init: schema present + user_version set, but no
schema_migrations table.
54 conn.executescript(_BASE)
55 conn.execute(f"PRAGMA user_version = {_VER}")
56 conn.execute("INSERT INTO documents (source, sha256) VALUES ('file', 'abc')")
57 migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
58 assert _VER in _applied(conn) # baseline recorded
59 assert conn.execute("SELECT count(*) FROM documents").fetchone()[0] == 1 # data
survived
60
61
62 def test_forward_migration_applies_once_and_bumps_version():
63 m = migrations.Migration(
64 version=_VER + 1, name="add_demo_col", sql="ALTER TABLE documents ADD COLUMN
demo TEXT"
65)
66 with dbmod.connect(":memory:") as conn:
67 v = migrations.run_migrations(
68 conn, baseline_sql=_BASE, baseline_version=_VER, migrations=[m]
69)
70 assert v == _VER + 1
71 assert conn.execute("PRAGMA user_version").fetchone()[0] == _VER + 1
72 assert "demo" in {r[1] for r in conn.execute("PRAGMA table_info(documents)")}
73 # Re-run must NOT re-ALTER (that would raise "duplicate column").
74 v2 = migrations.run_migrations(
75 conn, baseline_sql=_BASE, baseline_version=_VER, migrations=[m]
76)
77 assert v2 == _VER + 1
78
79
80 def test_baseline_drift_is_detected():
81 with dbmod.connect(":memory:") as conn:
82 migrations.run_migrations(conn, baseline_sql=_BASE, baseline_version=_VER)
83 with pytest.raises(MigrationError): # baseline edited without a migration
84 migrations.run_migrations(
85 conn, baseline_sql=_BASE + "\n-- edited\n", baseline_version=_VER
86)
87
88
89 def test_runs_inside_an_encrypted_connection(tmp_path):
90 pytest.importorskip("sqlcipher3")
91 key = "00112233445566778899aabbccddeeff00112233445566778899aabbccddeeff"
92 with dbmod.connect(tmp_path / "led.db", key=key) as conn:
93 dbmod.init_db(conn) # the production path through the runner, encrypted
94 assert _VER in {r[0] for r in conn.execute("SELECT version FROM
schema_migrations")}
95

```

### 31. user (2026-06-10T16:16:11.535Z)

```

tests/test_db_crypto_ops.py:34: def test_rekey_changes_the_key_and_keeps_data(tmp_path):

```

```

tests/test_db_crypto_ops.py:45:def test_rekey_with_wrong_current_key_raises(tmp_path):
tests/test_db_crypto_ops.py:55:def test_backup_is_encrypted_and_intact(tmp_path):
tests/test_db_crypto_ops.py:68:def test_backup_with_wrong_key_raises(tmp_path):
tests/test_db_crypto_ops.py:78:def test_encrypt_database_cleans_temp_on_failure(tmp_path,
monkeypatch):
tests/test_db_crypto_ops.py:95:def test_encrypted_ledger_handles_a_larger_table(tmp_path):
tests/test_db_crypto_ops.py:106:def
test_connect_encrypted_plaintext_message_points_to_db_encrypt(tmp_path):
tests/test_db_crypto_ops.py:127:def test_db_rotate_key_cli(tmp_path):
tests/test_db_crypto_ops.py:142:def test_db_backup_cli(tmp_path):
tests/test_db_crypto_ops.py:153:def
test_db_rotate_key_refuses_when_no_encrypted_ledger(tmp_path):
tests/test_recovery.py:17:def test_wrap_unwrap_roundtrip():
tests/test_recovery.py:22:def test_wrong_passphrase_is_rejected():
tests/test_recovery.py:28:def test_tampered_ciphertext_is_rejected():
tests/test_recovery.py:35:def test_short_passphrase_refused():
tests/test_recovery.py:40:def test_recovery_file_leaks_neither_key_nor_passphrase(tmp_path):
tests/test_recovery.py:48:def test_read_missing_or_garbage_file_raises(tmp_path):
tests/test_recovery.py:60:def test_cli_setup_recovery_then_recover(tmp_path, monkeypatch,
_isolate_keychain):
tests/test_recovery.py:89:def test_cli_setup_recovery_passphrase_mismatch(tmp_path, monkeypatch,
_isolate_keychain):
tests/test_recovery.py:101:def test_cli_recover_wrong_passphrase(tmp_path, monkeypatch,
_isolate_keychain):
tests/test_cli.py:11:def test_version_flag():
tests/test_cli.py:17:def test_info_lists_paths():
tests/test_cli.py:24:def test_ingest_bank_statement_unlocks_and_passes_password(tmp_path,
monkeypatch):
tests/test_cli.py:56:def test_db_status_on_missing_ledger(tmp_path):
tests/test_cli.py:63:def test_db_encrypt_migrates_and_status_reflects_it(tmp_path):
tests/test_cli.py:84:def test_ingest_bank_statement_unlock_failure_exits_nonzero(tmp_path,
monkeypatch):
tests/test_db_crypto.py:42:def test_encrypted_roundtrip_keeps_sql_queryability(tmp_path):
tests/test_db_crypto.py:57:def test_wrong_key_is_rejected(tmp_path):
tests/test_db_crypto.py:67:def test_on_disk_file_is_actually_encrypted(tmp_path):
tests/test_db_crypto.py:82:def test_memory_and_keyless_paths_stay_plaintext(tmp_path):
tests/test_db_crypto.py:94:def test_get_or_create_key_is_idempotent_and_64_hex(monkeypatch):
tests/test_db_crypto.py:106:def test_connect_encrypted_mints_key_and_roundtrips(tmp_path,
monkeypatch):
tests/test_db_crypto.py:117:def test_connect_encrypted_refuses_a_plaintext_ledger(tmp_path,
monkeypatch):
tests/test_db_crypto.py:129:def test_encrypt_database_migrates_in_place(tmp_path):
tests/test_db_crypto.py:146:def test_error_message_never_leaks_the_key(tmp_path):

docs/ROADMAP.md:46:1. **Regression CI for statement parsing** ? two-layer (synthetic cloud CI +
self-hosted real-statement
docs/ROADMAP.md:218: always-on (covers collaborators / when the self-hosted runner is
offline).
docs/ROADMAP.md:219:2. **A self-hosted runner that exercises the real statements** ? the
statements stay on the user's
docs/ROADMAP.md:244:### Layer 2 ? self-hosted runner for real statements (design)
docs/ROADMAP.md:245:- New workflow `.github/workflows/real-statements.yml`, `runs-on: self-
hosted`. Triggers on branch
docs/ROADMAP.md:249: `testdata/`), then runs `pytest -m local_statements`.
docs/ROADMAP.md:253:- The `local_statements` tests already assert what we want: HDFC reconcile
+ bookend; Axis/AU
docs/ROADMAP.md:261:### One-time setup the user must do (to actually run Layer 2)
docs/ROADMAP.md:262:1. Register a self-hosted Actions runner on the machine (Settings ? Actions
? Runners; Windows host).
docs/ROADMAP.md:268:- A self-hosted runner executes workflow code on the user's machine ? fine
for this **trusted private
docs/ROADMAP.md:270:- Layer 2 only runs when the machine is online; Layer 1 is the always-on
baseline.
docs/TESTING.md:143:2. **Real statements, local-only.** Behind `@pytest.mark.local_statements`,
reading `testdata/statements/` (gitignored). Skipped in CI; run by the user locally.

```

```
docs/TESTING.md:153:- **Default job** (`push` / `pr`): `setup-python@3.13` ? install ? `ruff`
check` ? `pytest -m "not ocr and not llm and not local_statements"`. Offline, no secrets.
Dependency cache on.
```

## 32. user (2026-06-10T16:16:21.607Z)

```
"""Hand-rolled, versioned, checksummed schema migrations (feedbackJune2026 P6).
```

```
Deliberately **not Alembic** ? muneem stays raw-`sqlite3` / `stdlib-light`. Migrations are an
ordered
list; each is applied exactly once, *inside* the (possibly encrypted) connection, and recorded
in a
`schema_migrations` table with a checksum. The schema-lock test
(`tests/test_db_invariants.py`)
pins the resulting shape, so accidental drift and deliberate changes both surface in review.
```

Model:

```
- **Baseline.** The current schema (`db._SCHEMA`, version `db.SCHEMA_VERSION`) is the *frozen*
baseline ? never edit it for new changes; add a migration instead. A fresh DB gets the
baseline
applied; a legacy DB created before this framework (tables present, no `schema_migrations`) is
detected and its baseline row back-filled (no DDL re-run).
- **Forward migrations.** Entries in `MIGRATIONS` with `version > baseline` run in order.
(None
yet ? the first lands with the unified-schema work, feedbackJune2026 P3.)
```

```
Known limitation (single-user, reviewed migrations): a crash *between* applying a migration's
DDL
and recording it could require manual cleanup; per-migration transactions can be added when the
first non-trivial migration lands.
"""
```

```
from __future__ import annotations
```

```
import hashlib
import sqlite3
from dataclasses import dataclass
```

```
from muneem.errors import MigrationError
```

```
_SCHEMA_MIGRATIONS = """
CREATE TABLE IF NOT EXISTS schema_migrations (
 version INTEGER PRIMARY KEY,
 name TEXT NOT NULL,
 checksum TEXT NOT NULL,
 applied_at TEXT NOT NULL DEFAULT (datetime('now'))
);
"""
```

```
@dataclass(frozen=True)
```

```
class Migration:
```

```
 """One forward migration: a version, a name, and the DDL to apply (run via
executescript)."""
```

```
 version: int
 name: str
 sql: str
```

```
 @property
```

```
 def checksum(self) -> str:
 return hashlib.sha256(self.sql.encode("utf-8")).hexdigest()
```



--- Forward migrations (version > baseline), applied in ascending order -----

**v5: provenance/confidence as a FIRST-CLASS but SEPARATE table (DESIGN §8.7, fee**  
**Keyed by document\_id (1:1) so every txn row reaches its provenance by the existing P**  
**per-row redundancy. source\_type readies it for non-PDF sources (aa/sms) we may ad**

```
_M5_PROVENANCE = """
CREATE TABLE IF NOT EXISTS provenance (
 id INTEGER PRIMARY KEY,
 document_id INTEGER NOT NULL UNIQUE REFERENCES documents(id) ON DELETE CASCADE,
 source_type TEXT NOT NULL,
 parser TEXT,
 parser_version TEXT,
 mapping_layer TEXT,
 confidence TEXT,
 created_at TEXT NOT NULL DEFAULT (datetime('now'))
);
"""
```

```
MIGRATIONS: list[Migration] = [
 Migration(version=5, name="provenance", sql=_M5_PROVENANCE),
]
```

```
def _checksum(sql: str) -> str:
 return hashlib.sha256(sql.encode("utf-8")).hexdigest()
```

```
def _user_tables(conn: sqlite3.Connection) -> list[str]:
 return [
 r[0]
 for r in conn.execute(
 "SELECT name FROM sqlite_master WHERE type='table' "
 "AND name NOT LIKE 'sqlite_%' AND name != 'schema_migrations'"
)
]
```

```
def _record(conn: sqlite3.Connection, version: int, name: str, checksum: str) -> None:
 conn.execute(
 "INSERT INTO schema_migrations (version, name, checksum) VALUES (?, ?, ?)",
 (version, name, checksum),
)
```

```
def run_migrations(
 conn: sqlite3.Connection,
 *,
 baseline_sql: str,
 baseline_version: int,
 migrations: list[Migration] | None = None,
) -> int:
 """Bring ``conn``'s schema up to date. Returns the resulting ``user_version``.
```

```
 Idempotent: re-running applies nothing already recorded. Raises :class:`MigrationError` if
 the
 frozen baseline's checksum no longer matches what was recorded (i.e. someone edited the
 baseline
 instead of adding a migration).
 """
```

```
 migrations = MIGRATIONS if migrations is None else migrations
 conn.executescript(_SCHEMA_MIGRATIONS) # bookkeeping table first (idempotent)
 applied = {r[0] for r in conn.execute("SELECT version FROM schema_migrations")}
```

```

current = conn.execute("PRAGMA user_version").fetchone()[0]
baseline_checksum = _checksum(baseline_sql)

if baseline_version in applied:
 recorded = conn.execute(
 "SELECT checksum FROM schema_migrations WHERE version = ?", (baseline_version,)
).fetchone()
 if recorded and recorded[0] != baseline_checksum:
 raise MigrationError(
=====
src/muneem/db.py:266:def rekey_database(db_path: Path | str, current_key: str, new_key: str) ->
None:
src/muneem/cli.py:428:@db_app.command("rotate-key")
src/muneem/cli.py:429:def db_rotate_key(
src/muneem/cli.py:447: typer.echo(f"rotated the encryption key for {Path(db_path).name}.")
src/muneem/db_key.py:30: _keychain.set_secret(_DB_KEY_NAME, key)
src/muneem/db_key.py:46: _keychain.set_secret(_DB_KEY_NAME, key)

```

### 33. user (2026-06-10T16:16:28.868Z)

```

from muneem import db as dbmod
from muneem.db_key import peek_key

if not Path(db_path).exists() or dbmod.is_plaintext_sqlite(db_path):
 typer.echo("no encrypted ledger here (run 'muneem db encrypt' first).", err=True)
 raise typer.Exit(1)
key = peek_key()
if not key:
 typer.echo("no encryption key in the keychain for this ledger.", err=True)
 raise typer.Exit(1)
return key

@db_app.command("rotate-key")
def db_rotate_key(
 db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data dir)."),
) -> None:
 """Rotate the ledger's encryption key (SQLCipher rekey) and update the keychain."""
 from muneem import db as dbmod
 from muneem import db_key

 db_path = db or load_config().db_path
 current = _require_encrypted_ledger(db_path)
 fresh = db_key.new_key()
 # Keychain-first with rollback: if the file rekey fails, the keychain still matches the
file.
 db_key.set_key(fresh)
 try:
 dbmod.rekey_database(db_path, current, fresh)
 except dbmod.EncryptedDatabaseError as exc:
 db_key.set_key(current) # roll back; the file is still under the old key
 typer.echo(str(exc), err=True)
 raise typer.Exit(1) from exc
 typer.echo(f"rotated the encryption key for {Path(db_path).name}.")

@db_app.command("backup")
def db_backup(
 dest: Path = typer.Argument(..., help="Destination path for the encrypted backup copy."),
 db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data dir)."),
) -> None:
 """Write an encrypted online-backup copy of the ledger (consistent even with WAL active)."""
 from muneem import db as dbmod

 db_path = db or load_config().db_path

```

```

key = _require_encrypted_ledger(db_path)
try:
 dbmod.backup_database(db_path, dest, key)
except dbmod.EncryptedDatabaseError as exc:
 typer.echo(str(exc), err=True)
 raise typer.Exit(1) from exc
typer.echo(f"backed up {Path(db_path).name} to {dest} (encrypted).")

```

```

@db_app.command("setup-recovery")
def db_setup_recovery(
 out: Path | None = typer.Option(
 """

```

## Keychain entry name (under the shared "muneem" service) holding the 64-hex-char D

```

_DB_KEY_NAME = "db-key"

```

```

def get_or_create_key() -> str:
 """Return the ledger's 256-bit key as 64 hex chars, minting+storing one on first use."""
 existing = _keychain.get_secret(_DB_KEY_NAME)
 if existing:
 return existing
 key = _stdlib_secrets.token_bytes(32).hex()
 _keychain.set_secret(_DB_KEY_NAME, key)
 return key

```

```

def peek_key() -> str | None:
 """Return the stored key, or ``None`` if none has been minted yet. Never creates one."""
 return _keychain.get_secret(_DB_KEY_NAME)

```

```

def new_key() -> str:
 """Generate a fresh random 256-bit key as 64 hex chars (not stored)."""
 return _stdlib_secrets.token_bytes(32).hex()

```

```

def set_key(key: str) -> None:
 """Overwrite the stored DB key ? used by key rotation, **after** the file has been
rekeyed."""
 _keychain.set_secret(_DB_KEY_NAME, key)

```

### 34. user (2026-06-10T16:16:43.705Z)

```

expiry_raw, expiry_type = text_fields.expiry_raw, text_fields.expiry_type

if ocr:
 from muneem.ocr import ocr_pdf, tesseract_available

 if not tesseract_available():
 typer.echo("warning: --ocr requested but Tesseract not found; skipping OCR.",
err=True)
 else:
 ocr_fields = extract_offer(ocr_pdf(pdf))
 known = {c for c, _ in found}
 for code in ocr_fields.codes:
 if code not in known:
 found.append((code, "ocr"))
 known.add(code)
 if expiry_raw is None:
 expiry_raw, expiry_type = ocr_fields.expiry_raw, ocr_fields.expiry_type

with _ledger(db_path) as conn:
 dbmod.init_db(conn)

```

```

existing = dbmod.get_document_by_sha(conn, ex.sha256)
if existing is not None:
 typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
 return
doc_id = dbmod.insert_document(
 conn,
 source="file",
 external_id=pdf.name,
 doc_type=doc_type,
 sha256=ex.sha256,
 path=str(pdf.resolve()),
 status="parsed",
)
for code, src in found:
 dbmod.insert_offer(
 conn,
 document_id=doc_id,
 promo_code=code,
 expiry_raw=expiry_raw,
 expiry_type=expiry_type,
 source=src,
 source_text=ex.text[:2000],
)

summary = ", ".join(f"{c} [{s}]" for c, s in found) if found else "none"
typer.echo(f"ingested {pdf.name} (doc {doc_id}); {len(found)} offer(s): {summary}")

```

```

@offers_app.command("list")
def offers_list(
 db: Path | None = typer.Option(None, help="SQLite path (defaults to the local data dir)."),
) -> None:
 """List extracted offers."""
 from muneem import db as dbmod

 db_path = db or load_config().db_path
 if not Path(db_path).exists():
 typer.echo("no offers yet (database not created).")
 return

 with _ledger(db_path) as conn:
 dbmod.init_db(conn)
 rows = dbmod.list_offers(conn)

 if not rows:
 typer.echo("no offers found.")
 return

 for row in rows:
 who = row["merchant"] or row["external_id"] or "?"
 when = f" ? expires {row['expiry_raw']}" if row["expiry_raw"] else ""
 tag = " [ocr ? verify]" if row["source"] == "ocr" else ""
 typer.echo(f"[{row['id']}] {who}: {row['promo_code']}{when}{tag}")

```

```

@txns_app.command("list")
def txns_list(
 bank: str = typer.Argument(..., help="Which bank's transactions to list (e.g. hdfc, axis)."),
) ==
 from muneem import db as dbmod

 db_path = db or load_config().db_path
 if not Path(db_path).exists():
 typer.echo("database not created yet.")
 return

```

```

bank = bank.lower()
if bank not in BANK_DISPATCH:
 typer.echo(f"Unknown bank: {bank}", err=True)
 return

list_fn, neg_key, pos_key = BANK_DISPATCH[bank]
with _ledger(db_path) as conn:
 dbmod.init_db(conn)
 rows = list_fn(conn, limit=limit)

if not rows:
 typer.echo(f"no {bank} transactions found.")
 return
for row in rows:
 if bank == "axis_cc":
 amount = f"+{row[pos_key]}" if row["type"] == "Cr" else f"-{row[neg_key]}"
 else:
 amount = f"+{row[pos_key]}" if row[pos_key] else f"-{row[neg_key]}"
 part = (row["particulars"] or "")[:50]
 typer.echo(f"[{row['account_number']}] {row['txn_date']} : {part:<50} | {amount:>10}")

===

 typer.echo(f"Could not unlock {pdf.name}: no working password{hint}.", err=True)
 raise typer.Exit(1)

ex = extract(pdf, password=password or "") # resolved above; `or ""` keeps the type str
parser = get_parser(ex.text)
if parser is None:
 typer.echo(f"No parser recognizes {pdf.name} as a supported bank statement.", err=True)
 raise typer.Exit(1)

with _ledger(db_path) as conn:
 dbmod.init_db(conn)
 existing = dbmod.get_document_by_sha(conn, ex.sha256)
 if existing is not None:
 typer.echo(f"already ingested (doc {existing['id']}): {pdf.name}")
 return
 doc_id = dbmod.insert_document(
 conn,
 source="file",
 external_id=pdf.name,
 doc_type=doc_type,
 sha256=ex.sha256,
from muneem.unlock import is_encrypted

password = ""
if is_encrypted(pdf):
 password = resolve_password(pdf, interactive=interactive) or ""
 if not password:
 return ("locked", 0, None)
try:
 ex = extract(pdf, password=password)
 parser = get_parser(ex.text)
except Exception as exc: # eval must not crash on one bad file; record the redacted reason
 _log.warning(
 "eval: extract/route failed for %s (%s)", redact_filename(pdf), type(exc).__name__
)
 return ("error", 0, None)
if parser is None:
 return ("no-parser", 0, None)
name = parser.__class__.__name__
try:
 with dbmod.connect(":memory:") as conn:
 dbmod.init_db(conn)
 doc_id = dbmod.insert_document(

```

```

 conn, source="file", external_id=pdf.name, sha256=ex.sha256, status="parsed"
)
 count = parser.parse_and_insert(pdf, conn, doc_id, password=password)
 doc = dbmod.get_document_by_sha(conn, ex.sha256)
 status = doc["status"] if doc else "parsed"
except Exception as exc:
 _log.warning(
 "eval: parse/store failed for %s (%s)", redact_filename(pdf), type(exc).__name__
)
 return ("error", 0, name)
return (status, count, name)

```

@app.command("eval")

### 35. user (2026-06-10T16:17:27.561Z)

```

.githubhooks
exit=0

6:``local_statements`` test runs the real parser on the user's PDFs (skipped in CI).
213:@pytest.mark.local_statements

tests/parsers/test_local_axis.py::test_local_axis_never_stores_unreconciled[AXIS BANK Statement
for May 2025.pdf]
tests/parsers/test_local_axis.py::test_local_axis_never_stores_unreconciled[AXIS BANK Statement
for October 2023.pdf]
tests/parsers/test_local_axis.py::test_local_axis_never_stores_unreconciled[AXIS BANK Statement
for October 2025.pdf]
tests/test_ocr.py::test_swiggy_image_code_recovered_fuzzily
tests/test_ocr.py::test_flipkart_ocr_yields_no_false_code
tests/test_unlock.py::test_real_hdfc_statements_decrypt

21/411 tests collected (390 deselected) in 0.85s

```

### 36. assistant (2026-06-10T16:18:34.013Z)

I have everything needed. Compiling the structured report.

### 37. user (2026-06-10T16:20:15.031Z)

Structured output provided successfully

### 38. assistant (2026-06-10T16:20:27.221Z)

Audit complete. The structured report has been returned: the offline suite passes (390 tests, 6.35s, 86.17% coverage vs the 80% gate), the claimed CI architecture is real, and the key findings center on the untested key-rotation failure window, plaintext real-data snapshots from a dead fixture, the absent real-statement CI layer, idealized synthetic fixtures, and the unblocked `client\_secret\*.json` filename pattern.